1 · MIPS Reference Data v2

Instruction formats (bit ranges) & where the 8 hex digits H0…H7 fall

bits	31–28 (H0)	27–24 (H1)	23–20 (H2)	19–16 (H3)	15–12 (H4)	11–8 (H5)	7–4 (H6)	3-0 (H7)
R op|rs|rt|rd|sh|fn 6|5|5|5|5|6	op[5:2]	op[1:0] · rs[4:3]	rs[2:0] · rt[4]	rt[3:0]	rd[4:1]	rd[0] · sh[4:2]	sh[1:0] · fn[5:4]	fn[3:0]
I op|rs|rt|imm 6|5|5|16	op[5:2]	op[1:0] · rs[4:3]	rs[2:0] · rt[4]	rt[3:0]	immediate[15:0] = exactly the last 4 hex digits (H4H5H6H7)			
J op|target 6|26	op[5:2]	op[1:0] · t[25:24]	target[23:0] = last 6 hex digits; jump address = {(PC+4)[31:28], target, 00}					

Core instruction table (opcode/funct in hex & binary)

instr	assembly form	op / funct	binary	operation (C-like)	notes
R-type: opcode = 0x00 = 000000 — instruction chosen by funct					
add	add rd,rs,rt	0 / 20	100000	rd = rs + rt	overflow → exception
addu	addu rd,rs,rt	0 / 21	100001	rd = rs + rt	no overflow exception
sub	sub rd,rs,rt	0 / 22	100010	rd = rs − rt	order matters
subu	subu rd,rs,rt	0 / 23	100011	rd = rs − rt	no overflow exc.
and	and rd,rs,rt	0 / 24	100100	rd = rs & rt	masking
or	or rd,rs,rt	0 / 25	100101	rd = rs | rt	force bits to 1
xor	xor rd,rs,rt	0 / 26	100110	rd = rs ^ rt	green sheet p2
nor	nor rd,rs,rt	0 / 27	100111	rd = ~(rs | rt)	NOT = nor rd,rs,$0
slt	slt rd,rs,rt	0 / 2A	101010	rd = (rs < rt) ? 1 : 0	signed
sltu	sltu rd,rs,rt	0 / 2B	101011	rd = (rs < rt) ? 1 : 0	unsigned
sll	sll rd,rt,sh	0 / 00	000000	rd = rt << sh	**rs = 0**; nop = 0x00000000
srl	srl rd,rt,sh	0 / 02	000010	rd = rt >>> sh (0-fill)	rs = 0
sra	sra rd,rt,sh	0 / 03	000011	rd = rt >> sh (sign-fill)	rounds toward −∞
sllv	sllv rd,rt,rs	0 / 04	000100	rd = rt << rs[4:0]	srlv 06, srav 07
jr	jr rs	0 / 08	001000	PC = rs	return: jr $ra
mult(u)	mult rs,rt	0 / 18,19	01100x	{Hi,Lo} = rs × rt	mfhi 0/10, mflo 0/12
div(u)	div rs,rt	0 / 1A,1B	01101x	Lo = rs/rt; Hi = rs%rt	
I-type: opcode identifies instruction; SE = sign-extend imm, ZE = zero-extend imm					
beq	beq rs,rt,L	04	000100	if(rs==rt) PC=PC+4+SE(imm)×4	1st operand = **rs**
bne	bne rs,rt,L	05	000101	if(rs!=rt) PC=PC+4+SE(imm)×4	imm counts words
addi	addi rt,rs,C	08	001000	rt = rs + SE(C)	C ∈ [−32768, 32767]
addiu	addiu rt,rs,C	09	001001	rt = rs + SE(C)	**still sign-extends**
slti	slti rt,rs,C	0A	001010	rt = (rs < SE(C)) ? 1 : 0	signed
sltiu	sltiu rt,rs,C	0B	001011	rt = (rs $<_u$ SE(C))	SE, then unsigned cmp
andi	andi rt,rs,C	0C	001100	rt = rs & ZE(C)	**upper 16 → 0**
ori	ori rt,rs,C	0D	001101	rt = rs | ZE(C)	C ∈ [0, 65535]
xori	xori rt,rs,C	0E	001110	rt = rs ^ ZE(C)	no nori exists
lui	lui rt,C	0F	001111	rt = C << 16 (low half = 0)	rs=0 → hex 3C0…/3C1…
lb	lb rt,C(rs)	20	100000	rt = SE(byte M[rs+SE(C)])	any address
lh	lh rt,C(rs)	21	100001	rt = SE(half M[..])	even address
lw	lw rt,C(rs)	23	100011	rt = M[rs+SE(C)]	addr % 4 == 0
lbu	lbu rt,C(rs)	24	100100	rt = ZE(byte M[..])	lhu = 25 (100101)
sb	sb rt,C(rs)	28	101000	byte M[..] = rt[7:0]	only low byte
sh	sh rt,C(rs)	29	101001	half M[..] = rt[15:0]	
sw	sw rt,C(rs)	2B	101011	M[rs+SE(C)] = rt	rt is a **source**
J-type					
j	j L	02	000010	PC = {(PC+4)[31:28], t26, 00}	same 256 MB block
jal	jal L	03	000011	$ra = return addr; PC = JumpAddr	sheet says PC+8*

*Green sheet's PC+8 assumes a branch-delay slot. CS2100 (Assg 1 note) uses **no delay slot** ⇒ the return address is simply the next instruction, PC+4.

Addressing modes in MIPS (L8) & which field is the operand

mode	operand / target	instructions	format
Register	value in rs / rt	add sub and or xor nor slt sll srl	R
Immediate	constant inside instruction	addi andi ori xori slti lui	I
Base (displacement)	M[rs + SE(imm)]	lw sw lb lbu lh lhu sb sh	I
PC-relative	(PC+4) + SE(imm)×4	beq bne	I
Pseudo-direct	{(PC+4)[31:28], t26, 00}	j jal	J

L10 says MIPS has **3** (operand) addressing modes: register, immediate, displacement. L8 adds the 2 control-flow modes ⇒ 5. Read the question's wording. "beq/bne: ignore PC bits, count displacement; j: use PC bits, ignore displacement."

How to read a row of the instruction table

table says	in a question it looks like	meaning
addi rt,rs,C	addi $t1, $sp, -120	rt = $t1, rs = $sp, C = −120 ⇒ $t1 = $sp + (−120)
ori rt,rs,C	ori $t0, $t0, 0xF0F0	$t0 = $t0 | 0x0000F0F0
lw rt,C(rs)	lw $v1, -100($t7)	address = $t7 + (−100); $v1 = the word there
lui rt,C	lui $t0, 0xAAAA	$t0 = 0xAAAA0000

Symbols: rs, rt, rd = register fields (source, target, destination) · **C** = the 16-bit constant (immediate) · **sh** = shift amount · **M[a]** = memory at address a · **byte/half** = 1 / 2 bytes of memory · **rt[7:0]** = bits 7…0 of rt (its lowest byte) · **|, &, ^, ~** = OR, AND, XOR, NOT · **<<** shift left, **>>>** logical right (0-fill), **>>** arithmetic right (sign-fill).
SE(C) = sign-extend to 32 bits (copy the top bit): SE(0xFF9C) = 0xFFFFFF9C = −100. **ZE(C)** = zero-extend (pad with 0s): ZE(0xFF9C) = 0x0000FF9C = 65436. Same 16 bits, different value — the instruction decides.
op / funct column: the opcode (or funct for R-type) in hex; **binary** column: the same number as 6 bits.

Why the numbers are what they are (L8 "strange facts") 32 registers ⇒ 5-bit register fields; shamt 5 bits because shifting ≥32 clears the register; imm = rd+shamt+funct = 5+5+6 = 16 bits; opcode/rs/rt stay in the same place in R and I so the hardware can read registers before knowing the format; instructions are word-aligned so branch/jump fields drop the two 0 LSBs (×4 range). No `subi` (use addi with −C), no `not` (nor with $zero), no `nori` (rarely needed), no real `blt` (slt + bne).

Registers (number → 5-bit field)

#	name	5-bit	hex	use	#	name	5-bit	hex	use
0	$zero	00000	00	always 0	16	$s0	10000	10	saved
1	$at	00001	01	asm temp	17	$s1	10001	11	saved
2	$v0	00010	02	result	18	$s2	10010	12	saved
3	$v1	00011	03	result	19	$s3	10011	13	saved
4	$a0	00100	04	arg	20	$s4	10100	14	saved
5	$a1	00101	05	arg	21	$s5	10101	15	saved
6	$a2	00110	06	arg	22	$s6	10110	16	saved
7	$a3	00111	07	arg	23	$s7	10111	17	saved
8	$t0	01000	08	temp	24	$t8	11000	18	temp
9	$t1	01001	09	temp	25	$t9	11001	19	temp
10	$t2	01010	0A	temp	26	$k0	11010	1A	OS
11	$t3	01011	0B	temp	27	$k1	11011	1B	OS
12	$t4	01100	0C	temp	28	$gp	11100	1C	global ptr
13	$t5	01101	0D	temp	29	$sp	11101	1D	stack ptr
14	$t6	01110	0E	temp	30	$fp	11110	1E	frame ptr
15	$t7	01111	0F	temp	31	$ra	11111	1F	return addr

$s0–$s7, $gp, $sp, $fp, $ra preserved across calls; $t, $a, $v not. $at = assembler temp (pseudo-instr.); $k0/$k1 = OS. Writing $zero has no effect.

Assembly order ≠ field order

- R: `op rd, rs, rt` → fields **rs | rt | rd**.
- Shifts: `sll rd, rt, sh` → **rs = 00000**, source in rt.
- sllv/srlv: `sllv rd, rt, rs` — amount is the *last* operand but sits in **rs**.
- I arith/logic: `op rt, rs, C` → destination is the **2nd** field (rt).
- Load/store: `op rt, C(rs)` → base = rs.
- Branch: `beq rs, rt, L` → first reg is **rs**. (Swapping them gives 0x1704… instead of 0x1498… — AY24/25 Q13 distractor.)
- `jr rs`: rt = rd = sh = 0, funct 08.

Extension rules (most-tested trap)

- **Sign-extend** imm: addi, addiu, slti, sltiu, all loads/stores (offset), beq/bne.
- **Zero-extend** imm: andi, ori, xori. lui: imm → upper half, lower 16 = 0.
- Loaded *data*: lb/lh sign-extend; lbu/lhu zero-extend.
- "u" in add**u**/add**iu**/sub**u** = **no overflow trap** only. "u" in slt**u**/slti**u**/lb**u**/lh**u**/mult**u**/div**u** = **unsigned**.
- SE(0x8000…0xFFFF) = 0xFFFF8000…0xFFFFFFFF = imm − 65536.
- Ranges: signed C16 $[-2^{15}, 2^{15}-1]$; logical C16 $[0, 2^{16}-1]$; shamt C5 [0, 31] (shift by 32 would just clear the register).

Memory & control-flow facts

- Byte-addressed; 32-bit addresses → 2^{30} words. Consecutive words differ by **4**.
- Word aligned ⇔ last hex digit ∈ {0,4,8,C}; half aligned ⇔ even.
- Branch reach: $\pm 2^{15}$ words from PC+4 → $[PC+4-2^{17}, PC+4+2^{17}-4]$ bytes.
- j reach: the 256 MB block [X000_0000, XFFF_FFFC] where X = top nibble of **PC+4**.
- $sp points to last **occupied** word; stack grows toward lower addresses.

Words you should recognise on sight

0x00000000	nop = sll $0,$0,0	0x03E00008	jr $ra
0x0000000C	syscall	0x00004020	add $t0,$0,$0
0x23BDFFFC	addi $sp,$sp,−4	0x27BDFFFC	addiu $sp,$sp,−4
0xAFBF0000	sw $ra,0($sp)	0x8FBF0000	lw $ra,0($sp)
0x2002000A	addi $v0,$0,10	0x3C01…	lui $at,… (pseudo)

2 · Decode & encode without converting in your head

look-up grids replace all 5-bit field arithmetic

Step 1 — first byte → instruction

byte	op	instr	byte	op	instr
00–03	00	R-type	**38–3B**	0E	xori
08–0B	02	j	**3C**	0F	lui
0C–0F	03	jal	**80–83**	20	lb
10–13	04	beq	**84–87**	21	lh
14–17	05	bne	**8C–8F**	23	lw
20–23	08	addi	**90–93**	24	lbu
24–27	09	addiu	**94–97**	25	lhu
28–2B	0A	slti	**A0–A3**	28	sb
2C–2F	0B	sltiu	**A4–A7**	29	sh
30–33	0C	andi	**AC–AF**	2B	sw
34–37	0D	ori	**44–47**	11	FP

First hex digit alone
0 R-type (00–03), j (08–0B), jal (0C–0F)
1 beq (10–13), bne (14–17)
2 addi, addiu, slti, sltiu
3 andi, ori, xori, lui (3C)
8 lb (80–83), lh, lw (8C–8F)
9 lbu, lhu **A** sb, sh, sw (AC–AF)

Step 4 — R-type funct → name

funct	name	funct	name
00	sll	20	add
02	srl	21	addu
03	sra	22	sub
04/06/07	sllv/srlv/srav	23	subu
08	jr	24	and
09	jalr	25	or
0C	syscall	26	xor
10/12	mfhi/mflo	27	nor
18/19	mult/multu	2A	slt
1A/1B	div/divu	2B	sltu

Reading funct from the last two hex digits
Rule 1 — not a shift: the last two digits **are** the funct. …20 → add, …2A → slt, …08 → jr.
Rule 2 — last two digits not in the table? It's a shift (sll/srl/sra). The 2nd-last digit (H6) is shared between shift amount and funct:

```
H6 = 8 = 10 | 00
         ^^   ^^
       shift  funct
```

Keep only the **last 2 bits** of H6:

H6 is	binary ends in	keep
0 4 8 C	…00	**0**
1 5 9 D	…01	**1**
2 6 A E	…10	**2**
3 7 B F	…11	**3**

funct = kept number, then the last digit:

```
...80: 8 -> 0, then 0 -> 0x00 = sll
...C2: C -> 0, then 2 -> 0x02 = srl
...43: 4 -> 0, then 3 -> 0x03 = sra
```

(0x00084080 = sll $t0,$t0,2 · 0x000840C2 = srl $t0,$t0,3 · 0x00094943 = sra $t1,$t1,5)

Step 2 — registers straight from the hex digits (H0 = leftmost)

rs is built from the last 2 bits of H1 and the first 3 bits of H2. **Row:** find your H1 digit in the row labels. **Column:** find your H2 digit in the column headers. Read the register where they meet.

H1 digit ↓ / H2 digit →	0/1	2/3	4/5	6/7	8/9	A/B	C/D	E/F
H1 = 0, 4, 8, C	$zero$_0$	at_1$	$v0$_2$	$v1$_3$	$a0$_4$	$a1$_5$	$a2$_6$	$a3$_7$
H1 = 1, 5, 9, D	$t0$_8$	$t1$_9$	$t2$_{10}$	$t3$_{11}$	$t4$_{12}$	$t5$_{13}$	$t6$_{14}$	$t7$_{15}$
H1 = 2, 6, A, E	$s0$_{16}$	$s1$_{17}$	$s2$_{18}$	$s3$_{19}$	$s4$_{20}$	$s5$_{21}$	$s6$_{22}$	$s7$_{23}$
H1 = 3, 7, B, F	$t8$_{24}$	$t9$_{25}$	$k0$_{26}$	$k1$_{27}$	gp_{28}$	sp_{29}$	fp_{30}$	ra_{31}$

rt = H2 even/odd (top bit) + H3:

H2 ↓ / H3 →	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
even	$zero	$at	$v0	$v1	$a0	$a1	$a2	$a3	$t0	$t1	$t2	$t3	$t4	$t5	$t6	$t7
odd	$s0	$s1	$s2	$s3	$s4	$s5	$s6	$s7	$t8	$t9	$k0	$k1	$gp	$sp	$fp	$ra

rd (R-type only) = H4 + whether H5 ≥ 8:

H5 ↓ / H4 →	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0–7	$zero	$v0	$a0	$a2	$t0	$t2	$t4	$t6	$s0	$s2	$s4	$s6	$t8	$k0	$gp	$fp
8–F	$at	$v1	$a1	$a3	$t1	$t3	$t5	$t7	$s1	$s3	$s5	$s7	$t9	$k1	$sp	$ra

shamt = last 3 bits of H5 followed by first 2 bits of H6 (non-zero only for shifts; …4080: 000 + 10 = 00010 = 2) **imm** = H4H5H6H7, a 16-bit **2's complement** number (p.4): H4 = 0–7 ⇒ positive, read as is; H4 = 8–F ⇒ negative: value = (unsigned value) − 65536 (0x10000 = 2^{16}), e.g. FF9C = 65436 − 65536 = −100. **j target** = last 2 bits of H1 followed by H2…H7 (26 bits).
The grids work backwards for **encoding**: e.g. rs = $sp (29) is in the row "H1 = 3, 7, B, F" and column C/D ⇒ H1 is one of 3, 7, B, F (which one depends on the opcode), and H2 = C if rt < 16, D if rt ≥ 16.

Worked decodes (all exam values)

```
0x8DE3FF9C   8DE8C-8F -> lw     (AY25/26 Q13)
  H1=D: row 1,5,9,D; H2=E: col E/F -> rs=$t7
  H2=E even, H3=3: rt=$v1
  imm FF9C (16-bit 2's comp, H4=F -> negative)
    invert+1: 0063+1 = 0x64 = 100 -> -100
    (1's comp would give -99: wrong) [p.3, p.4]
  => lw $v1, -100($t7)

0x00516020   00 -> R-type           (AY25/26 Q20-23)
  H1=0: row 0,4,8,C; H2=5: col 4/5 -> rs=$v0
  H2 odd, H3=1: rt=$s1 (17)
  H4=6, H5=0(<8): rd=$t4 (12)
  shamt=0, funct = last 2 digits = 0x20 (no shift)
  => add $t4, $v0, $s1

0x12C3FBAD   12 -> beq              (AY25/26 Q14)
  H1=2: row 2,6,A,E; H2=C: col C/D -> rs=$s6
  H2=C even, H3=3: rt=$v1
  imm FBAD (2's comp, negative) [p.4]
    invert+1: 0452+1 = 0x453 -> -1107 words
  PC=0x5204: 0x5208 - 1107*4 = 0x40BC [x4: p.2B]

0x10000103   10 -> beq $0,$0,0x103 (AY24/25 Q28)
  PC=0x491: 0x495 + 0x40C = 0x8A1 (taken)
```

Worked encodes

```
addi $t1,$sp,-120                   (AY24/25 Q12)
 op 001000 rs 11101 rt 01001
 -120 in 16-bit 2's comp [p.4]:
   0x10000-0x78 = 0xFF88
 0010 0011 1010 1001 -> 0x23A9FF88

bne $a0,$t8,EXIT  PC=0x401C, EXIT=0x35F0 (Q13)
 imm = (0x35F0-0x4020)/4 = -0xA30/4
     = -0x28C -> 2's comp 0xFD74 [p.4]
 000101 00100 11000 -> 0x1498 -> 0x1498FD74

lw $v1,-100($t7) -> 100011 01111 00011
 1000 1101 1110 0011 = 8DE3 -> 0x8DE3FF9C

sw $ra,-8($sp) -> 0xAFBFFFF8
sll $t0,$t0,2  -> 000000 00000 01000 01000
                  00010 000000 = 0x00084080
```

Branch & jump arithmetic (full step-by-step walkthrough on **p.2B**)

- **Branch taken:** PC′ = (PC + 4) + SE(imm)×4. Not taken: PC + 4.
- **Encode:** imm = (Target − (PC+4)) / 4 = number of instructions to skip, counted from the instruction *after* the branch. Backward ⇒ negative.
- **j:** PC′ = ((PC+4) & 0xF0000000) | (target26 × 4). Encode: target26 = (Target & 0x0FFFFFFF)/4, legal only if Target and PC+4 share the top nibble.
- **Block-edge trap** (AY24/25 Q15): j 0x08012348 at PC 0x2FFFFFFC → PC+4 = 0x30000000 → 0x30048D20 (not 0x2…).
- **j ↔ beq equivalence** (AY25/26 Q16): 0x080000AD at 0x1014 jumps to 0xAD×4 = 0x2B4; same as beq $v0,$v0,−857 since (0x2B4 − 0x1018)/4 = −857. bne $zero,$zero,.. never branches.
- imm = 0xFFFF ⇒ target = PC itself (infinite loop if taken); imm = 0 ⇒ branch to PC+4 (no-op).

Hex speed tricks

- **×4:** shift left 2. Per digit (with carries): 1→4, 2→8, 3→C, 4→10, 8→20, C→30, F→3C. 0x012348×4 = 0x048D20.
- **÷4 / negative:** byte differences between instructions always end in 0,4,8,C.
- **−X in 16 bits** (step by step on **p.4B**) = 0x10000 − X (or invert each digit F−d, then +1): −0x28C → FD73+1 = **FD74**.
- **Read SE(imm):** FFxx = xx − 0x100 (e.g. FF88 = 0x88−256 = −120); Fxxx = xxx − 0x1000.
- Add PC + negative offset in 32 bits = subtract the magnitude: 0x5208 − 0x114C = 0x40BC.

Pseudo-instructions → real instructions (only real ones count for performance)

pseudo	expansion
move d,s	add d,s,$zero
Also valid: addu/or d,s,$zero, addi d,s,0, ori d,s,0, sll d,s,0. **Not** andi d,s,0xFF (zero-extends ⇒ wipes upper bits, AY25/26 Q15).	
li d,C (16-bit)	addi d,$zero,C or ori d,$zero,C
li d,C32 / la d,L	lui d,C[31:16] ; ori d,d,C[15:0]
not d,s	nor d,s,$zero
neg d,s	sub d,$zero,s
clear d	add d,$zero,$zero
b L / beqz s,L	beq $zero,$zero,L / beq s,$zero,L
blt a,b,L	slt $at,a,b ; bne $at,$zero,L
bge a,b,L	slt $at,a,b ; beq $at,$zero,L
bgt a,b,L	slt $at,b,a ; bne $at,$zero,L
ble a,b,L	slt $at,b,a ; beq $at,$zero,L
push x / pop x	addi $sp,$sp,-4 ; sw x,0($sp) / lw x,0($sp) ; addi $sp,$sp,4
far beq a,b,L	bne a,b,Skip ; j L ; Skip:
lui x,C (no lui)	ori x,$zero,C ; sll x,x,16 (AY24/25 Q26)

push must decrement **first** (AY24/25 Q16 ⇒ B). sw x,-4($sp) alone never moves $sp; lw in a push is backwards.

Building 32-bit constants (lui + ?)

```
lui $t0,0x1234 ; ori  $t0,$t0,0x8000  -> 0x12348000  OK
lui $t0,0x1234 ; xori $t0,$t0,0x8000  -> 0x12348000  OK (low half was 0)
lui $t0,0x1234 ; addi $t0,$t0,0x8000  -> 0x12338000  WRONG (SE = -32768)
lui $t0,0x1234 ; andi ...             -> low half stays 0 / upper wiped
```

AY24/25 Q14 printed option D as **xori** — that *does* work (x⊕0 = x). The answer key lists D as **nor** and says E. If your paper says xori, D is correct.

sllv as a pseudo-instruction (AY25/26 Q26)

```
       andi $at, $rz, 0x1F   # amount = low 5 bits of $rz
       add  $rx, $ry, $zero  # copy value
LOOP:  beq  $at, $zero, OUT
       sll  $rx, $rx, 1
       addi $at, $at, -1
       j    LOOP
OUT:
```

Draft key's first line says $rx — typo for $rz. Order matters: read $rz into $at *before* writing $rx (in case rx = rz). Executes 2 + 4k + 1 instructions for amount k.

Syscalls (labs; $v0 = code, then `syscall`)

1 print int ($a0)	4 print string ($a0 = addr)	5 read int → $v0
8 read string ($a0 buf, $a1 len)	10 exit	11 print char ($a0)

2B · Branch & jump arithmetic, step by step

where does the program go next? · every example checked

Two facts everything else follows from

1. Every instruction is **4 bytes**, so the next instruction is always at **PC + 4**. The CPU computes PC + 4 first, and branches are measured **from PC + 4**, not from the branch itself.
2. A branch immediate counts **instructions**, not bytes. Instructions → bytes: **×4**. Bytes → instructions: **÷4**.

The immediate is a 16-bit **2's complement** number (SE = sign-extend): first hex digit 8–F ⇒ negative ⇒ the branch goes **backwards** (conversion: p.4B).

1. Branch taken: PC′ = (PC + 4) + imm × 4

Start at the next instruction, then move imm instructions forward (or back if imm is negative). **Not taken:** just PC + 4.

```
Forward, imm = 3:
 0x1000  beq $t0,$t1,L   <- PC (the branch)
 0x1004  ...             <- PC+4: counting starts here
 0x1008  ...               skipped
 0x100C  ...               skipped
 0x1010  L: ...          <- 0x1004 + 3*4 = 0x1010

Backward (a loop), imm = -3 = 0xFFFD:
 0x2000  L: add ...      <- 0x200C - 3*4 = 0x2000
 0x2004  ...
 0x2008  bne $t0,$0,L    <- PC
 0x200C  ...             <- PC+4
```

2. Decoding a branch — recipe

1. Take the last 4 hex digits (the immediate).
2. First digit 0–7: positive, convert. First digit 8–F: negative — F−d, +1, convert, add a minus sign (p.4B).
3. ×4 to get bytes.
4. Add to **PC + 4** (subtract if negative).

```
AY25/26 Q14: beq at PC = 0x5204, imm = 0xFBAD
 1. imm = FBAD
 2. F -> negative: F-d = 0452, +1 = 0453 = 1107 -> -1107
 3. x4 in hex: -0x453 * 4 = -0x114C bytes
      3*4 = C
      5*4 = 14 -> write 4, carry 1
      4*4 = 10, +1 = 11
      [ref: p.3 x4 column, p.2 Hex speed tricks]
 4. PC+4 = 0x5208;  0x5208 - 0x114C = 0x40BC   (C)
```

3. Encoding a branch: imm = (Target − (PC + 4)) / 4

1. Compute PC + 4.
2. Target − (PC + 4) = distance in bytes (negative if the target is earlier).
3. ÷4 = distance in instructions.
4. Negative? Write it as 16-bit 2's complement (pad to 4 digits, F−d, +1).

```
AY24/25 Q13: bne $a0,$t8,EXIT
  PC     = 0x401C  (address of the bne itself)
  Target = EXIT = 0x35F0  (where it should go)

Step 1 | PC+4: 0x401C + 4 = 0x4020
Step 2 | Target - (PC+4) = 0x35F0 - 0x4020
       | Target is smaller -> result is negative,
       | so compute the bigger minus the smaller:
       |    4 0 2 0
       |  - 3 5 F 0
       |  right to left, like decimal:
       |   col 1: 0-0 = 0
       |   col 2: 2-F, borrow: (2+16)-15 = 3
       |   col 3: 0-1(lent)-5, borrow: 16-6 = A
       |   col 4: 4-1(lent)-3 = 0   => 0x0A30
       | then add the minus: -0xA30 bytes
Step 3 | /4 (bytes -> instructions):
       | A30 = 1010 0011 0000  (p.3 table)
       | /4 = drop last 2 bits: 10 1000 1100
       | regroup in 4s: 0010 1000 1100 = 2 8 C
       |    = 0x28C  -> -0x28C instructions
Step 4 | negative -> 16-bit 2's comp (p.4B sec. E):
       | pad 028C -> F-d -> FD73 -> +1 -> FD74
Assemble | op bne 000101, rs $a0 00100,
         | rt $t8 11000, imm FD74
         | 0001 0100 1001 1000 = 1498
         | => 0x1498FD74   (C)
```

Counting shortcut: imm = number of instructions from PC+4 up to (not including) the target. Target directly after the branch ⇒ imm = 0; one instruction skipped ⇒ imm = 1.

4. Jump (j / jal) — the exam shortcut

Decode: where does this j go?

1. **×4 the whole instruction word** (hex, digit by digit, p.3 ×4 column).
2. **Replace the first hex digit** with the first hex digit of **PC + 4**.

Done. (The opcode lands in the first digit after ×4 — that's the digit you throw away.)

question	word ×4	PC+4 first digit	target
0x08100040 at 0x00400000	~~2~~0400100	0	**0x00400100**
0x080000AD at 0x1014 (AY25/26 Q16)	~~2~~00002B4	0	**0x000002B4**
0x08012348 at 0x2FFFFFFC (AY24/25 Q15)	~~2~~0048D20	**3** (0x30000000)	**0x30048D20**

Encode: j to a target address

1. **Can it reach?** Target's first hex digit must equal PC+4's first hex digit. If not ⇒ j cannot do it.
2. **Replace the target's first digit with 2** (for jal: **3**).
3. **÷4** ⇒ that is the instruction word.

0x00400100 → 0x**2**0400100 ÷ 4 = **0x08100040** · jal: 0x**3**0400100 ÷ 4 = **0x0C100040**

÷4 in hex = long division like decimal; a remainder carries into the next digit as ×16: 2÷4 = 0 r2 → (2·16+0)÷4 = 8 → 4÷4 = 1 → 0 → 0 → 1÷4 = 0 r1 → (16+0)÷4 = 4 → 0 ⇒ 0x08100040. Or: binary, drop the last 2 bits, regroup in 4s.

5. The block-edge trap (AY24/25 Q15)

```
j 0x08012348 at PC = 0x2FFFFFFC
 word x4 = 0x20048D20
 PC+4 = 0x30000000    <- first digit rolled over 2 -> 3!
 target = 0x30048D20   (not 0x20048D20)
```

Only the **last** instruction of a 256 MB block (address xFFFFFFC) does this. A j at 0x1FFFFFF8 has PC+4 = 0x1FFFFFFC (block 1); a j at 0x1FFFFFFC has PC+4 = 0x20000000 (block 2).

6. j ↔ beq equivalence (AY25/26 Q16)

```
0x080000AD at PC = 0x1014
 j target: word x4 = 0x200002B4; PC+4 = 0x1018,
           first digit 0 -> jumps to 0x000002B4
   x4: D*4=34 -> write 4, carry 3;
       A*4=28, THEN +3 = 2B -> write B, carry 2
 same as a branch with imm = (0x2B4 - 0x1018)/4
                           = (692 - 4120)/4 = -857
 -> beq $v0, $v0, -857   (A)
```

- `beq $x,$x,L` (same register twice) is **always taken**: a register always equals itself. That's how a branch imitates a jump.
- `bne $zero,$zero,L` asks "0 ≠ 0?" — **never taken**.
- A branch can only replace a j if the target is within ±32768 instructions of PC+4.

7. Special immediates & limits

imm	value	target	meaning
0x0000	0	PC + 4	same as not branching
0x0001	+1	PC + 8	skips exactly one instruction
0xFFFF	−1	PC	branches to itself: infinite loop if taken
0x7FFF	+32767	PC + 0x20000	furthest forward
0x8000	−32768	PC − 0x1FFFC	furthest backward

Branch reach ≈ ±128 KB around the branch. j reach = anywhere in the same 256 MB block (same first hex digit as PC+4). Target too far for a branch? Invert it and jump: `bne a,b,Skip ; j L ; Skip:`.

Mistakes that lose the mark

- Measuring from **PC** instead of **PC + 4** (off by one instruction).
- Forgetting ×4 when decoding, or ÷4 when encoding (bytes vs instructions).
- Treating 0xFBAD as positive — first digit 8–F means backwards.
- j: taking the first digit from PC instead of PC + 4 (the block-edge trap).
- j: keeping the first digit of word×4 (that digit is opcode, always replace it).

Self-test (cover the answers)

question	answer
bne at 0x00400010, imm 0x0005: target?	0x00400014 + 20 = 0x00400028
beq at 0x00400030, imm 0xFFF8: target?	−8: 0x00400034 − 32 = 0x00400014
beq at 0x00400100 to 0x004000F0: imm?	(0xF0 − 0x104)/4 = −5 → 0xFFFB
beq at 0x1000 with imm 0xFFFF: target?	0x1000 (itself)
Encode j to 0x00400100	0x08100040
Can a j at 0x1FFFFFFC reach 0x10000000?	No — PC+4 = 0x20000000, block 2

3 · Number tables — hex ↔ binary ↔ every representation

read values, don't compute them

One hex digit, every interpretation (4-bit)

hex	binary	u	2's	1's	S-M	xs-8	F−d (NOT)	×4 (hex)	2|2	3|1	1|3
0	0000	0	0	0	0	−8	F	0	0|0	0|0	0|0
1	0001	1	1	1	1	−7	E	4	0|1	0|1	0|1
2	0010	2	2	2	2	−6	D	8	0|2	1|0	0|2
3	0011	3	3	3	3	−5	C	C	0|3	1|1	0|3
4	0100	4	4	4	4	−4	B	10	1|0	2|0	0|4
5	0101	5	5	5	5	−3	A	14	1|1	2|1	0|5
6	0110	6	6	6	6	−2	9	18	1|2	3|0	0|6
7	0111	7	7	7	7	−1	8	1C	1|3	3|1	0|7
8	1000	8	−8	−7	−0	0	7	20	2|0	4|0	1|0
9	1001	9	−7	−6	−1	1	6	24	2|1	4|1	1|1
A	1010	10	−6	−5	−2	2	5	28	2|2	5|0	1|2
B	1011	11	−5	−4	−3	3	4	2C	2|3	5|1	1|3
C	1100	12	−4	−3	−4	4	3	30	3|0	6|0	1|4
D	1101	13	−3	−2	−5	5	2	34	3|1	6|1	1|5
E	1110	14	−2	−1	−6	6	1	38	3|2	7|0	1|6
F	1111	15	−1	−0	−7	7	0	3C	3|3	7|1	1|7

u unsigned · **2's/1's/S-M** as a 4-bit signed number · **xs-8** excess-8 (= 2's with MSB flipped) · **F−d** bitwise NOT of the digit (1's complement in hex) · **×4** digit shifted left 2 (branch/jump offsets) · **2|2, 3|1, 1|3** how the digit splits across field boundaries (H1 = op|rs is 2|2, H2 = rs|rt is 3|1, H5 = rd|shamt is 1|3, H6 = shamt|funct is 2|2).

Reading bigger numbers from the table

- **Hex → binary:** one row per digit. 0xBEA0 = 1011 1110 1010 0000.
- **Binary → hex:** group 4 from the right (integers) or outward from the binary point (fractions); pad with 0s.
- **NOT / 1's complement in hex:** F−d per digit: ~0x00FF_1234 = 0xFF00_EDCB.
- **2's complement (negate) in hex:** F−d per digit, then +1: −0x0064 = 0xFF9B+1 = 0xFF9C.
- **Signed value of a 2's-comp word:** leading digit 8–F ⇒ value = unsigned − 2^n.
- **Octal:** group 3 bits: 1011 1110 = 010 111 110 = 276_8.

Powers of 2

n	2^n	hex
0	1	1
1	2	2
2	4	4
3	8	8
4	16	10
5	32	20
6	64	40
7	128	80
8	256	100
9	512	200
10	1,024	400
11	2,048	800
12	4,096	1000
13	8,192	2000
14	16,384	4000
15	32,768	8000
16	65,536	10000

n	2^n	hex
16	65,536	10000
17	131,072	20000
18	262,144	40000
19	524,288	80000
20	1,048,576	100000
21	2,097,152	200000
22	4,194,304	400000
23	8,388,608	800000
24	16,777,216	1000000
25	33,554,432	2000000
26	67,108,864	4000000
27	134,217,728	8000000
28	268,435,456	10000000
29	536,870,912	20000000
30	1,073,741,824	40000000
31	2,147,483,648	80000000
32	4,294,967,296	100000000
32	4,294,967,296	1_0000_0000

Byte table 0x00–0xFF: unsigned (top) and 8-bit 2's complement (red, rows 8–F)

	_0	_1	_2	_3	_4	_5	_6	_7	_8	_9	_A	_B	_C	_D	_E	_F
0_	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1_	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
2_	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47
3_	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63
4_	64	65	66	67	68	69	70	71	72	73	74	75	76	77	78	79
5_	80	81	82	83	84	85	86	87	88	89	90	91	92	93	94	95
6_	96	97	98	99	100	101	102	103	104	105	106	107	108	109	110	111
7_	112	113	114	115	116	117	118	119	120	121	122	123	124	125	126	127
8_	128 −128	129 −127	130 −126	131 −125	132 −124	133 −123	134 −122	135 −121	136 −120	137 −119	138 −118	139 −117	140 −116	141 −115	142 −114	143 −113
9_	144 −112	145 −111	146 −110	147 −109	148 −108	149 −107	150 −106	151 −105	152 −104	153 −103	154 −102	155 −101	156 −100	157 −99	158 −98	159 −97
A_	160 −96	161 −95	162 −94	163 −93	164 −92	165 −91	166 −90	167 −89	168 −88	169 −87	170 −86	171 −85	172 −84	173 −83	174 −82	175 −81
B_	176 −80	177 −79	178 −78	179 −77	180 −76	181 −75	182 −74	183 −73	184 −72	185 −71	186 −70	187 −69	188 −68	189 −67	190 −66	191 −65
C_	192 −64	193 −63	194 −62	195 −61	196 −60	197 −59	198 −58	199 −57	200 −56	201 −55	202 −54	203 −53	204 −52	205 −51	206 −50	207 −49
D_	208 −48	209 −47	210 −46	211 −45	212 −44	213 −43	214 −42	215 −41	216 −40	217 −39	218 −38	219 −37	220 −36	221 −35	222 −34	223 −33
E_	224 −32	225 −31	226 −30	227 −29	228 −28	229 −27	230 −26	231 −25	232 −24	233 −23	234 −22	235 −21	236 −20	237 −19	238 −18	239 −17
F_	240 −16	241 −15	242 −14	243 −13	244 −12	245 −11	246 −10	247 −9	248 −8	249 −7	250 −6	251 −5	252 −4	253 −3	254 −2	255 −1

Row = high digit, column = low digit. Other 8-bit readings of rows 8–F: **1's comp** = −(255 − u) = red value + 1; **sign-magnitude** = −(u − 128); **excess-128** = u − 128 (for every row); **excess-127** (IEEE exponent) = u − 127. Example 0x9C = 156 → 2's −100, 1's −99, SM −28, xs-128 +28. lb of byte 0x9C gives 0xFFFFFF9C = −100; lbu gives 156.

Negative powers (fractions)

n	decimal	binary
2^{-1}	0.5	0.1
2^{-2}	0.25	0.01
2^{-3}	0.125	0.001
2^{-4}	0.0625	0.0001
2^{-5}	0.03125	0.00001
2^{-6}	0.015625	0.000001
2^{-7}	0.0078125	0.0000001
2^{-8}	0.00390625	0.00000001
2^{-9}	0.001953125	0.000000001
2^{-10}	0.0009765625	0.0000000001
2^{-11}	0.00048828125	0.00000000001
2^{-12}	0.000244140625	0.000000000001

Constants worth knowing

0x7FFFFFFF	2,147,483,647 (INT_MAX)
0x80000000	−2,147,483,648 (INT_MIN)
0xFFFFFFFF	−1 / 4,294,967,295
0x00007FFF	32,767 (max imm)
0xFFFF8000	−32,768 = SE(0x8000)
0x0000FFFF	65,535 (max ZE imm)
0x01000000	2^{24} = 16,777,216
0x10000000	2^{28} = 256 MB (j block)
0x00020000	2^{17} = branch reach (bytes)
0x64 / 0x3E8	100 / 1000
0x400 / 0x1000	1024 / 4096
0x3F800000	1.0f

ASCII (printable 0x20–0x7E) — for lb/sb/string questions

	_0	_1	_2	_3	_4	_5	_6	_7	_8	_9	_A	_B	_C	_D	_E	_F
2_	SP	!	"	#	$	%	&	'	(	)	*	+	,	-	.	/
3_	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4_	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5_	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6_	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7_	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	DEL

'\0' = 0x00, '\t' = 0x09, '\n' = 0x0A, '\r' = 0x0D. '0' = 0x30, 'A' = 0x41, 'a' = 0x61. Upper ↔ lower differ only in bit 5 (0x20): toupper = c − 32 = c & 0xDF; tolower = c | 0x20. Digit value = c − '0' = c & 0x0F. All ASCII ≤ 0x7F, so lb and lbu agree on them; bytes ≥ 0x80 differ.

4 · Integer representations — full guide

n bits	Unsigned	Sign-magnitude	1's complement	2's complement	Excess-K ($K = 2^{n-1}$)
value of pattern	$\Sigma\, b_i 2^i$	MSB = sign, rest = magnitude	MSB 1 ⇒ −(invert all bits)	$-b_{n-1}2^{n-1} + \Sigma_{i<n-1} b_i 2^i$	u − K
range	0 … 2^n-1	$-(2^{n-1}-1)$ … $+(2^{n-1}-1)$	$-(2^{n-1}-1)$ … $+(2^{n-1}-1)$	-2^{n-1} … $2^{n-1}-1$	-2^{n-1} … $2^{n-1}-1$
8-bit range	0…255	−127…127	−127…127	−128…127	−128…127
zeros	1	2: 0000 0000, 1000 0000	2: 0000 0000, 1111 1111	1	1 (at 1000 0000)
negate	—	flip MSB	invert every bit	invert every bit, +1	stored′ = 2K − u (i.e. 2's-negate the pattern)
sign-extend	pad 0s on the left	**sign stays at MSB, pad 0s after it**	replicate MSB	replicate MSB	re-bias: v + K′
add	add	same sign: add magnitudes; else subtract smaller from larger, sign of larger	add; carry-out of MSB is added back to LSB (**end-around carry**)	add; **discard** carry-out	add patterns, then subtract K
overflow	carry out of MSB	carry out of the magnitude	operands have the same sign and the result has the other sign (2's: $C_{in}(MSB) \neq C_{out}(MSB)$)		—
−5 in 8 bits	—	`1000 0101`	`1111 1010`	`1111 1011`	`0111 1011`
1001 1100	156	−28	−99	−100	+28

2's complement — everything

- **Read a negative value:** value = unsigned − 2^n (use the byte table), or negate and put a minus sign: 1110 1100 → 0001 0100 = 20 ⇒ −20.
- **Negate shortcut:** copy bits from the right up to and including the first 1, invert everything left of it: 0110 1000 → 1001 1000.
- **Hex negate:** F−d per digit, +1. −0x0064 = 0xFF9C. −0x00000001 = 0xFFFFFFFF.
- **The odd one out:** 1000…0 = -2^{n-1} has no positive partner; negating it gives itself (overflow).
- **Subtract:** A − B = A + ~B + 1. That is exactly the ALU: Binvert = 1 and carry-in = 1 (why ALUcontrol for sub is 0110).
- **Overflow only when adding like signs** (or subtracting unlike signs). A carry-out by itself is **not** overflow:
  `1111 1100 (−4) + 1111 1010 (−6) = 1|1111 0110 = −10` ✓ (carry discarded)
  `0110 0100 (100) + 0011 0010 (50) = 1001 0110 = −106` ✗ overflow
  `6-bit: 011010 (26) + 010011 (19) = 101101 = −19` ✗ overflow
- **Sign extension** preserves value: 1010 (−6) → 1111 1010 (−6). In hex: prefix F…F when the leading digit is 8–F.
- **Largest positive with odd widths:** a 0 sign bit then all 1s. 137 bits = 1 + 34×4 ⇒ top hex digit holds only the sign bit ⇒ `0x0` followed by 34 F's (AY24/25 Q2 ⇒ A).
- **Shifts:** sll k = ×2^k (overflow if bits leaving differ from the new sign). sra k = ⌊x/2^k⌋ (rounds toward −∞: −7 sra 1 = −4), but C's −7/2 = −3. srl on a negative gives a huge positive.
- **Multiply by constants with shifts:** ×5 = (x<<2)+x = (x<<1)+(x<<1)+x; ×7 = (x<<3)−x; ×9 = (x<<3)+x; ×10 = (x<<3)+(x<<1); ×15 = (x<<4)−x. AY25/26 Q3: "shift 3 then subtract X" gives ×7 ⇒ the one that does **not** work (C).
- **slt vs sltu:** $a = 0xFFFFFFFF, $b = 1: slt → 1 (−1 < 1), sltu → 0 (4294967295 > 1).

1's complement — everything

- **Read:** MSB 0 ⇒ as unsigned. MSB 1 ⇒ invert all bits, put a minus sign. Equivalently −(2^n−1−u). 1111 1010 → −0000 0101 = −5.
- **Negate:** invert every bit (hex: F−d per digit).
- **Two zeros:** 0000 0000 = +0 and 1111 1111 = −0.
- **Add with end-around carry** (carry out of MSB is added to the result):
  `0000 0111 (7) + 1111 1101 (−2) = 1|0000 0100 → +1 = 0000 0101 (5)`
  `1111 0011 (−12) + 0000 1001 (9) = 1111 1100 (−3)`, no carry
  `1111 1111 (−0) + 0000 0001 = 1|0000 0000 → +1 = 0000 0001`
- **Subtract:** A + ~B, then end-around carry.
- **Overflow:** same-sign operands giving an opposite-sign result (check after the end-around carry).
- **Relations:** 1's(x) = (2^n−1) − x; 2's(x) = 2^n − x = 1's(x) + 1. For the same negative value, the 1's pattern = 2's pattern − 1.
- **Sign extension:** replicate MSB. 4-bit 1010 (−5) → 1111 1010 (−5). The *pattern* is the same as 2's-comp extension of 1010 (−6 → −6).
- **Radix generalisation:** (r−1)'s complement = (r^n−1) − x (each digit → r−1−d); r's complement = r^n − x. 4-digit decimal: 9's comp of 0123 = 9876, 10's = 9877.

Sign-magnitude — traps

- **Sign extension is NOT replication:** 1010_{sm} (−2) → **1000 0010** (AY25/26 Q1 ⇒ A). 1111 1010 would be −122.
- **Arithmetic:** do it in sign + magnitude. AY24/25 Q1: 0011 0010 − 1011 1101 = (+50) − (−61) = +111 = **0110 1111** (C).
- **Range of n bits:** ±(2^{n-1}−1). 9 bits ⇒ ±255, so +290 is impossible (AY25/26 Q2 ⇒ C).

Excess-K (biased)

- stored = value + K. Usual K = 2^{n-1} ⇒ pattern = 2's complement with the **MSB flipped**. IEEE 754 uses K = 2^{n-1}−1 (127 for 8 bits).
- Patterns sort in the same order as values (why exponents use it: compare floats like integers).
- Excess-8, 4-bit: 0000 = −8, 1000 = 0, 1111 = +7. Excess-128: 0111 1011 = −5.

Base conversion (worked, step-by-step versions on p.4B)

- **Base r → decimal:** $\Sigma\, d_i r^i$. Use Horner on a calculator: $3a79_{12}$ = ((3·12+10)·12+7)·12+9 = 6717.
- **Decimal → base r (integer):** repeatedly ÷ r, read remainders bottom-up.
- **Fraction → base r:** repeatedly × r, read the integer parts top-down. 0.6875 → 1.375, 0.75, 1.5, 1.0 ⇒ 0.1011. 13.625 = 1101.101.
- **Non-terminating:** 0.1_{10} = 0.0 0011 0011 0011…$_2$ (why the Patriot clock drifted, L9).
- **Binary ↔ octal/hex:** group 3/4 bits outward from the binary point, padding the ends with 0s.

Unknown-base puzzles (Assg 1 Q2 / AY24/25 Q3 / AY25/26 Q4)

1. **Prune by digits:** base > largest digit used. Digit a = 10 ⇒ base ≥ 11; a 9 ⇒ base ≥ 10. This kills options instantly.
2. Convert each side to decimal with Horner and compare. You must test every option: "which is NOT valid" ⇒ exactly one fails.
3. **Solve algebraically** when one base is unknown: $121_b = 144 \Rightarrow b^2+2b+1 = (b+1)^2 = 144 \Rightarrow b = 11$. $31_b = 25 \Rightarrow 3b+1 = 25 \Rightarrow b = 8$.

AY24/25 Q3: $3a79_{13} - 2035_{15} = 8381 - 6800 = 1581 \neq 1792_{11} = 2279$ ⇒ D. AY25/26 Q4: $4090_{11} - 1520_{13} = 5423 - 3068 = 2355 = 3206_9$ ⇒ D.

Fixed point

- n bits, k fraction bits: value = (integer reading of the pattern) / 2^k. Precision = 2^{-k} (constant everywhere, unlike floats).
- 2's-comp range: [-2^{n-1-k}, $2^{n-1-k} - 2^{-k}$]. 8 bits, k = 3: 1110 1100 = −20/8 = **−2.5**.

Same value, every 8-bit encoding (check yourself)

value	S-M	1's	2's	excess-128
+20	`0001 0100`	`0001 0100`	`0001 0100`	`1001 0100`
−20	`1001 0100`	`1110 1011`	`1110 1100`	`0110 1100`
−1	`1000 0001`	`1111 1110`	`1111 1111`	`0111 1111`
0	`±0: 0000 0000, 1000 0000`	`0000 0000, 1111 1111`	`0000 0000`	`1000 0000`
−127	`1111 1111`	`1000 0000`	`1000 0001`	`0000 0001`
−128	—	—	`1000 0000`	`0000 0000`

Range & precision (L9 definitions)

- **Range** = the set of distinct representable values. The four interesting ones: largest positive, most negative, smallest positive, smallest-magnitude negative.
- **Overflow** = beyond the largest. **Underflow** = a non-zero value smaller in magnitude than the smallest representable (not an issue for integers).
- **Precision** = minimum non-zero distance between two representable values: integer 1, fixed point 2^{-k}, floating point relative (depends on the exponent).

4B · Step-by-step conversions & complement recipes

A. Place values — the only numbers you need

hex place	16^4	16^3	16^2	16^1	16^0
value	65536	4096	256	16	1

d	1	2	3	4	5	6	7	8
d×16	16	32	48	64	80	96	112	128
d×256	256	512	768	1024	1280	1536	1792	2048

d	9	A	B	C	D	E	F
d×16	144	160	176	192	208	224	240
d×256	2304	2560	2816	3072	3328	3584	3840

B. Hex → decimal

Multiply each digit by its place value and add.

```
0x3E8  = 3×256 + E(14)×16 + 8
       = 768 + 224 + 8 = 1000
0x2B4  = 2×256 + B(11)×16 + 4
       = 512 + 176 + 4 = 692
0x453  = 4×256 + 5×16 + 3
       = 1024 + 80 + 3 = 1107
0xFF9C = 15×4096 + 15×256 + 9×16 + 12
       = 61440 + 3840 + 144 + 12 = 65436  (unsigned)
```

Calculator shortcut (Horner): start with the first digit, then repeatedly ×16 and add the next digit: 0x3E8 → 3 → 3×16+14 = 62 → 62×16+8 = 1000.

C. Decimal → hex

Method 1: divide by 16, read remainders bottom-up.

```
1000 ÷ 16 = 62  remainder 8    ↑ read
  62 ÷ 16 =  3  remainder 14=E  ↑ upwards
   3 ÷ 16 =  0  remainder 3     ↑       => 0x3E8

65416 ÷ 16 = 4088 r 8
 4088 ÷ 16 =  255 r 8
  255 ÷ 16 =   15 r 15 = F
   15 ÷ 16 =    0 r 15 = F               => 0xFF88
```

Method 2: peel off place values (quicker for small numbers, using table A).

```
857 : 3×256 = 768, left 89
      5×16  =  80, left 9          => 0x359
120 : 7×16  = 112, left 8          => 0x78
```

D. Binary ↔ hex ↔ decimal

- **Binary → hex:** group 4 bits *from the right*, pad the left with 0s, look up each group (p.3 table). 10 1101 → 0010 1101 = **0x2D**.
- **Hex → binary:** each digit becomes exactly 4 bits. 0xB6 = 1011 0110.
- **Binary → decimal:** go via hex: 1011 0110 = 0xB6 = 11×16 + 6 = **182**. Or add powers of 2: 128+32+16+4+2 = 182.
- **Decimal → binary:** go via hex: 45 = 2×16 + 13 = 0x2D = 0010 1101. Or subtract powers of 2: 45 = 32+8+4+1 ⇒ 101101.
- **Fractions:** ×2 repeatedly, take the integer parts top-down: 0.6875 → 1.375, 0.75, 1.5, 1.0 ⇒ 0.1011 (full method on p.4).

Self-test (cover the right column)

question	answer
200 → hex	0xC8 (12×16 + 8)
0xA5 → decimal	165 (160 + 5)
0x7D0 → decimal	2000 (1792 + 208)
0x1F4 → decimal	500 (256 + 240 + 4)
−75, 16-bit 2's comp	004B → FFB4 → FFB5
−300, 16-bit 2's comp	012C → FED3 → FED4
−32, 8-bit 2's comp	20 → DF → E0
−9, 8-bit 1's comp	09 → F6 (no +1)
FFE0 (16-bit 2's) → decimal	0xE0 = 224; 224 − 256 = −32
FEDC (16-bit 2's) → decimal	0123+1 = 0124 = 292 ⇒ −292
0x80, 8-bit 2's → decimal	128 − 256 = −128
0x80, 8-bit 1's → decimal	F−d: 7F = 127 ⇒ −127

E. 2's complement: decimal → hex (for immediates)

Recipe for −N in a W-bit field

1. Convert N (without the minus) to hex (section C).
2. **Pad with 0s to the full width first:** 16 bits = 4 digits, 8 bits = 2, 32 bits = 8.
3. Replace every digit d with **F − d** (the "F−d" column on p.3).
4. **Add 1** to the last digit (F + 1 = 0, carry 1 left).

Positive numbers: just convert and pad with 0s. Nothing else.

```
-120 (16-bit): 120 = 0x78 -> pad 0078
               F-d  -> FF87  -> +1 -> FF88
-1107 (AY25/26 Q14): 1107 = 0x453 -> 0453
               F-d  -> FBAC  -> +1 -> FBAD
-857 (AY25/26 Q16):  857 = 0x359  -> 0359
               F-d  -> FCA6  -> +1 -> FCA7
-100:          0064 -> FF9B -> +1 -> FF9C
-16 (carry):   0010 -> FFEF -> +1 -> FFF0
               (F+1 = 0 carry 1, E+1 = F)
-256 (2 carries): 0100 -> FEFF -> +1 -> FF00
-1:            0001 -> FFFE -> +1 -> FFFF
-32768:        8000 -> 7FFF -> +1 -> 8000 (!)
```

Other routes to the same answer: **0x10000 − N** in hex, or **65536 − N** in decimal then convert (65536 − 120 = 65416 = 0xFF88). Change the width by changing the constant: 8-bit uses 256, 32-bit uses 2^{32}.

F. 2's complement: hex → decimal

Look at the first digit. 0–7 ⇒ positive: convert normally (section B). 8–F ⇒ negative: use either method.

pattern	A: F−d, +1, convert, add −	B: unsigned − 2^W	value
FF9C (16)	0063+1 = 0064 = 100	65436 − 65536	**−100**
FBAD (16)	0452+1 = 0453 = 1107	64429 − 65536	**−1107**
FFF8 (16)	0007+1 = 0008	65528 − 65536	**−8**
8000 (16)	7FFF+1 = 8000 = 32768	32768 − 65536	**−32768**
F6 (8)	09+1 = 0A = 10	246 − 256	**−10**
FFFFFF00 (32)	000000FF+1 = 0100 = 256	—	**−256**

FFxx shortcut (16-bit): value = 0xxx − 256. FF88 → 0x88 = 136 − 256 = **−120**. FF9C → 156 − 256 = −100. **FFFx**: value = x − 16 (FFF8 = 8 − 16 = −8).

G. 1's complement: same recipe, but NO +1

Recipe −N: convert N, pad to width, F−d every digit. Stop. **Reading** a pattern whose first digit is 8–F: F−d every digit, convert, put a minus sign. Or unsigned − (2^W − 1), e.g. 65535 for 16 bits.

```
-120 (16-bit): 0078 -> F-d -> FF87        (2's comp: FF88)
-120 (8-bit):  78   -> F-d -> 87          (2's comp: 88)
-1   (16-bit): 0001 -> FFFE               (2's comp: FFFF)
read FF87:     F-d -> 0078 = 120 -> -120; or 65415 - 65535 = -120
zero has two patterns: 0000 (+0) and FFFF (-0)
add 7 + (-2):  0007 + FFFD = 1|0004 -> carry wraps: 0004+1 = 0005
```

H. The same negative number in every system (8-bit hex)

value	−1	−5	−20	−100	−120	−127	−128
sign-magnitude	81	85	94	E4	F8	FF	—
1's complement	FE	FA	EB	9B	87	80	—
2's complement	FF	FB	EC	9C	88	81	80

Pattern: **2's = 1's + 1** for every negative. Sign-magnitude just sets the top bit on the positive number (+0x80).

I. Changing width (sign / zero extension in hex)

- **2's or 1's complement:** first digit 8–F ⇒ pad with F; 0–7 ⇒ pad with 0. −120: 8-bit 88 → 16-bit FF88 → 32-bit FFFFFF88. +120: 78 → 0078.
- **Sign-magnitude:** move the sign bit to the new top, pad 0s after it: 8-bit F8 (−120) → 16-bit **8078**.
- **Zero-extend** (unsigned; andi/ori/xori; lbu/lhu): always pad with 0: 88 → 0088 (= 136, not −120).

Mistakes that give the wrong answer

- **Not padding before inverting:** −120 from "78" gives 88 (8-bit), not FF88. Pad to 4 digits first for 16-bit.
- **Forgetting +1** gives 1's complement (FF87), off by one.
- **Adding 1 before inverting:** 0078 → 0079 → FF86, wrong.
- **Reading FF9C as positive** (65436) in a signed field: check the first digit!
- **Mixing widths:** 0x88 is −120 in 8 bits but +136 in 16 bits (0x0088).

5 · IEEE 754 single precision (float) — layout, decode, encode

value = $(-1)^S \times 1.F \times 2^{E-127}$ · precision, rounding & AI formats on p.5B

A. Where each field sits — and which hex digit holds which bits (example 0xBEA00000)

hex digit	H0 = B				H1 = E				H2 = A				H3 = 0				H4 = 0				H5 = 0				H6 = 0				H7 = 0			
bit #	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
bit value	1	0	1	1	1	1	1	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
field	S	E = exponent (8 bits)								F = fraction (23 bits)																						

S (1 bit): 0 = positive, 1 = negative. **E** (8 bits): biased exponent, true exponent e = E − 127. **F** (23 bits): the bits after the binary point in 1.F (the leading 1 is not stored — "hidden").

Hex digits vs fields: H0 = S + first 3 bits of E · H1 = next 4 bits of E · H2 = last bit of E + first 3 bits of F · H3–H7 = rest of F. The exponent spans the **first three hex digits**, which is why the shortcut table (section D) looks at 3 digits.

B. Decode: hex → decimal

```
0xBEA00000                          (AY24/25 Q5)
Step 1 | hex -> binary, 4 bits per digit (p.3):
       | B    E    A    0    0 ...
       | 1011 1110 1010 0000 0000 ...
Step 2 | split 1 | 8 | 23:
       | S = 1
       | E = 0111 1101
       | F = 010 0000 0000 ...
Step 3 | S = 1 -> negative
Step 4 | E = 0111 1101 = 0x7D = 125
       | e = 125 - 127 = -2
Step 5 | 1.F = 1.010 (binary)
       |     = 1 + 0.25 = 1.25   (weights below)
Step 6 | value = -1.25 x 2^-2 = -1.25 / 4
       |       = -0.3125   (D)
```

Fraction bit weights (Step 5) 1st F bit = 0.5 · 2nd = 0.25 · 3rd = 0.125 · 4th = 0.0625 · 5th = 0.03125. First 3 F bits → 1.F: 000 = 1.0, 001 = 1.125, 010 = 1.25, 011 = 1.375, 100 = 1.5, 101 = 1.625, 110 = 1.75, 111 = 1.875.

C. Encode: decimal → hex

```
-12.5                                -> ?
Step 1 | sign: negative -> S = 1
Step 2 | 12.5 -> binary (p.4 conversions):
       | 12 = 1100, .5 = .1  ->  1100.1
Step 3 | normalise: point after the first 1
       | 1100.1 = 1.1001 x 2^3
       | (point moved 3 left -> e = +3)
Step 4 | E = e + 127 = 3 + 127 = 130
       | why +127? e can be negative
       |   (0.25 = 1.0 x 2^-2), but E is 8 plain
       |   bits (0-255, no sign). So store e
       |   shifted up by 127 (excess-127, p.4).
       |   Decode undoes it: e = E - 127.
       | 130 = 128 + 2 = 0x82 = 1000 0010
Step 5 | F = only the bits AFTER the point:
       |   1.1001 -> F starts 1001
       | the "1." is always there, so it is
       |   not stored (hidden bit)
       | pad 0s on the RIGHT to 23 bits
       |   (like 0.5 = 0.500, value unchanged):
       |   1001 0000 0000 0000 0000 000
Step 6 | line up S | E | F:
       | 1 | 1000 0010 | 1001 0000 ...
Step 7 | regroup in 4s -> hex (p.3):
       | 1100 0001 0100 1000 0000 ...
       |  C    1    4    8    0 ...
       | = 0xC1480000
```

Point moved **left** (number ≥ 2) ⇒ e positive; moved **right** (number < 1) ⇒ e negative:

```
0.15625                          -> 0x3E200000
Step 1 | positive -> S = 0
Step 2 | 0.15625 = 0.00101
Step 3 | 1.01 x 2^-3  (point moved 3 RIGHT)
Step 4 | E = -3 + 127 = 124 = 0111 1100
Step 5 | F = 01, then 0s -> 0100 0000 ...
Step 6 | 0 | 0111 1100 | 0100 0000 ...
Step 7 | 0011 1110 0010 0000 ... = 0x3E200000
```

D. Shortcut: read the exponent from the first 3 hex digits

Find the word's first 3 hex digits in a range below → read e. Example 0x**BEA**00000: BEA is in BE8–BEF ⇒ negative, e = −2. Then the **3rd digit** also gives the first 3 fraction bits: if the range ends in 8–F subtract 8 (A − 8 = 2 = 010), otherwise use it as is.

word starts (+)	word starts (−)	E	E binary	e = E−127	2^e
3B8–3BF	BB8–BBF	119	0111 0111	**−8**	1/256
3C0–3C7	BC0–BC7	120	0111 1000	**−7**	1/128
3C8–3CF	BC8–BCF	121	0111 1001	**−6**	0.015625
3D0–3D7	BD0–BD7	122	0111 1010	**−5**	0.03125
3D8–3DF	BD8–BDF	123	0111 1011	**−4**	0.0625
3E0–3E7	BE0–BE7	124	0111 1100	**−3**	0.125
3E8–3EF	BE8–BEF	125	0111 1101	**−2**	0.25
3F0–3F7	BF0–BF7	126	0111 1110	**−1**	0.5
3F8–3FF	BF8–BFF	127	0111 1111	**0**	1
400–407	C00–C07	128	1000 0000	**+1**	2
408–40F	C08–C0F	129	1000 0001	**+2**	4
410–417	C10–C17	130	1000 0010	**+3**	8
418–41F	C18–C1F	131	1000 0011	**+4**	16
420–427	C20–C27	132	1000 0100	**+5**	32
428–42F	C28–C2F	133	1000 0101	**+6**	64
430–437	C30–C37	134	1000 0110	**+7**	128
438–43F	C38–C3F	135	1000 0111	**+8**	256
440–447	C40–C47	136	1000 1000	**+9**	512
448–44F	C48–C4F	137	1000 1001	**+10**	1024

Special values (L9)

E bits	F bits	meaning
all 0	all 0	±0 (0x00000000, 0x80000000)
all 0	≠ 0	denormal: $(-1)^S \times \mathbf{0}.F \times 2^{\mathbf{-126}}$ (1 − bias; no hidden 1)
1…254	any	normalised: $1.F \times 2^{E-127}$
all 1	all 0	±∞ (0x7F800000, 0xFF800000)
all 1	≠ 0	NaN (e.g. 0x7FC00000; 0/0, ∞−∞)

Hex constants

0x3F800000	1.0	0xBF800000	−1.0 (AY25/26 Q5)
0x3F000000	0.5	0x40000000	2.0
0x3FC00000	1.5	0x3F400000	0.75
0x40400000	3.0	0x41200000	10.0
0x42C80000	100.0	0xC1480000	−12.5
0xBEA00000	−0.3125	0x3E200000	0.15625
0x3DCCCCCD	≈0.1 (inexact)	0x40490FDB	≈π
0x00800000	min normal $2^{-126} \approx 1.18e{-38}$	0x7F7FFFFF	max normal ≈ 3.40e38
0x00000001	min denormal $2^{-149} \approx 1.4e{-45}$	0x007FFFFF	max denormal

AY24/25 Q4: smallest positive **normalised** = E = 1, F = 0 = **0x00800000** (B). 0x00000001 is the smallest *denormal* — not normalised.

IEEE traps

- Bias is 127, so E = 0x7F means e = 0; E = 0x80 means e = +1. Don't forget the hidden 1 (normalised only).
- Denormal example: 0x00400000 = $0.1_2 \times 2^{-126} = 2^{-127}$. Exponent is −126, not −127.
- Normalisation gives one pattern per value (no duplicates) — except ±0.
- 0x80000000: as a float −0.0; as an int -2^{31}. Same bits, the instruction decides.
- Positive floats compare like unsigned ints (sign, then biased exponent, then fraction) — the point of excess-127.
- −x only flips bit 31: 1.0 → 0xBF800000. Doubling adds 1 to E: +0x00800000.

5B · Floats continued: integers, rounding, AI formats (L9)

continues p.5

A. Which integers can a float hold exactly? (AY24/25 Q6, AY25/26 Q6)

Why some can't: a float keeps only **24 significant bits**: the hidden 1 plus 23 bits of F. Normalising an integer puts the point right after its highest 1, and every bit after the point must fit in F's **23 slots**. A 24th bit falls off the end.

```
2^24 + 1  (25 bits long)
  1 0000 0000 0000 0000 0000 0001
normalise:
  1.0000 0000 0000 0000 0000 0001 x 2^24
    |- 24 bits after the point -|
  F has 23 slots -> the final 1 falls off
  stored: 1.000...000 x 2^24 = 2^24   NOT exact

2^24 + 2
  1 0000 0000 0000 0000 0000 0010
  the bit that falls off is a 0 -> nothing lost
  stored: 2^24 + 2                    exact
```

So above 2^{24} the **last bit must be 0** (even numbers only). Each time the number doubles in size, one more bit falls off:

Fast rule: which range is it in?

integer between	in hex	exact only if a multiple of	check the last hex digit
0 … 2^{24} (16,777,216)	≤ 0x1000000	**anything**	always exact
2^{24} … 2^{25}	0x1000000–0x1FFFFFF	**2**	even: 0 2 4 6 8 A C E
2^{25} … 2^{26}	0x2000000–0x3FFFFFF	**4**	0 4 8 C
2^{26} … 2^{27}	0x4000000–0x7FFFFFF	**8**	0 8
2^{27} … 2^{28}	0x8000000–0xFFFFFFF	**16**	0

In hex: a **7-digit** number starting with **1** must be even; starting with 2–3, a multiple of 4; 4–7, a multiple of 8; 8–F, end in 0.

```
AY24/25 Q6: smallest positive integer NOT exact?
  everything up to 2^24 is exact;
  the first odd number above 2^24 is 2^24 + 1
  = 16,777,217 = 0x1000001   (C)

AY25/26 Q6: X = 0x0120101C
  7 digits, starts with 1 -> must be even
  X   ends in C -> even -> exact
  X+1 ends in D -> odd  -> NOT exact,
      rounds to X        -> f(X) = f(X+1)   (B)
```

Alternative (the spread test): write it in binary; if (position of highest 1) − (position of lowest 1) ≤ 23 it is exact. Same answer, slower. Gap between neighbouring floats doubles each power of 2 (1 below 2^{24}, 2 up to 2^{25}, …); about 7–8 significant decimal digits.

B. Rounding (L9)

- Hardware keeps **3 extra bits** while computing: **guard**, **round**, **sticky** (sticky = 1 if any bit further right is 1). Extra bits appear when lining up exponents for addition, or when multiplying (n × n bits → 2n bits).
- Four modes: **round to nearest** (default), toward 0 (just chop), toward +∞, toward −∞.
- Round to nearest, **tie** (exactly halfway) ⇒ pick the neighbour whose last bit is 0 ("ties to even").

true value	neighbours (gap 2)	stored as	why
$2^{24}+1$	2^{24}, $2^{24}+2$	2^{24}	tie → even
$2^{24}+2$	—	$2^{24}+2$	exact
$2^{24}+3$	$2^{24}+2$, $2^{24}+4$	$2^{24}+4$	tie → even

- **Associativity is lost** because every step rounds: FP32 ($10^8 + -10^8$) + 1 = **1**, but 10^8 + (-10^8 + 1) = **0** (the +1 vanishes next to 10^8). Adding a list in a different order can give a different sum.
- **Patriot missile (1991):** time counted in 0.1 s ticks; 0.1 is not exact in binary (24-bit fixed point); after 100 h the error was ≈ 0.34 s ⇒ missed intercept, 28 killed.

C. Any float format, the same 3 rules

For a format with **k exponent bits** and **m fraction bits** (IEEE-style):

quantity	rule	FP16 (k=5, m=10)
bias	$2^{k-1} - 1$	$2^4 - 1$ = **15**
1.0	S = 0, E = bias, F = 0	0 01111 0000000000 = **0x3C00**
max normal	E = all 1s but last, F = all 1s: $(2 - 2^{-m}) \times 2^{bias}$	E = 11110 (e = 15): $1.11…1 \times 2^{15}$ = **65504** = 0x7BFF
min normal	E = 1: 2^{1-bias}	2^{-14} ≈ **6.1e−5**
min denormal	$2^{1-bias-m}$	2^{-24}
integers exact up to	2^{m+1}	2^{11} = **2048** (2049 is not)

BF16 = simply the **top 16 bits of an FP32** (same 8-bit exponent, fraction cut to 7 bits): FP32 1.0 = 0x3F80~~0000~~ ⇒ BF16 1.0 = **0x3F80**. Same range as FP32, far less precision.

D. INT8 quantisation (L9 slide 17), step by step

```
values: -12.47, 0, -7.6, 5.72, 9.65, -10.12, 8.31
Step 1 | largest magnitude: max|x| = 12.47
Step 2 | scale = 127 / 12.47 = 10.18
       | (127 = biggest INT8; INT4 uses 7)
Step 3 | q = round(x * scale):
       |  -12.47 -> -127      5.72 -> 58.3 -> 58
       |   -7.6  -> -77.4 -> -77
       |   9.65 -> 98,  -10.12 -> -103,  8.31 -> 85
Step 4 | to get back: x ~ q / scale
       |   58 / 10.18 = 5.70  (was 5.72: small error)
```

INT4 is less accurate than FP4 but runs on ordinary integer hardware; FP4 needs special hardware. Block-scaling variants (MXFP8/4, NVFP4, HiF8/4) apply scale factors at several levels.

Formats for the AI era (L9) — IEEE-style rules for any k-exponent / m-fraction format

format	bits	exp	frac	bias	+1.0	max normal*	min normal	ints exact up to	use (L9)
FP64	64	11	52	1023	0x3FF0000000000000	1.8e308	2.23e-308	2^{53}	scientific
FP32	32	8	23	127	0x3F800000	3.4e38	1.18e-38	2^{24}=16777216	general
BF16	16	8	7	127	0x3F80	3.39e38	1.18e-38	2^8=256	AI training
FP16	16	5	10	15	0x3C00	65504	6.1e-5	2^{11}=2048	graphics/AI
FP8 E4M3	8	4	3	7	0x38	240	0.015625	2^4=16	weights/activ.
FP8 E5M2	8	5	2	15	0x3C	57344	6.1e-5	2^3=8	gradients
FP4 E2M1	4	2	1	1	0x2	3	1	2^2=4	experimental

bias = $2^{k-1}-1$ · max normal = $(2-2^{-m})\cdot 2^{(2^k-2)-bias}$ · min normal = 2^{1-bias} · min denormal = $2^{1-bias-m}$ · integers exact up to 2^{m+1}. *Real FP8-E4M3 (OCP spec) reuses the all-1s exponent ⇒ max 448; real FP4 has no ∞ ⇒ max 6. **BF16 = the top 16 bits of FP32** (same 8-bit exponent, fraction cut to 7 bits): 1.0 = 0x3F80. FP16 1.0 = 0x3C00, max 65504 = 0x7BFF.

6 · C: pointers, arrays, strings, structs, scope — exam traps

every output below was compiled with gcc and run

Precedence you actually need (high → low)

1	`x++ x-- f() a[i] s.m p->m`	postfix — binds tightest
2	`++x --x *p &x ! ~ -x (type) sizeof`	prefix/unary, right-to-left
3	`* / %`	arithmetic
4	`+ -`	
5	`<< >>`	shifts
6	`< <= > >=  then == !=`	relational
7	`& then ^ then |`	bitwise (below ==!)
8	`&& then || then ?: then = += …`	logical, ternary, assign

Consequences: `*p++` = `*(p++)`; `*p.m` = `*(p.m)` (so write `p->m` or `(*p).m`); `x & 1 == 0` = `x & (1==0)` = 0 always.

The four pointer-increment forms

`int a[] = {10,20,30}; int *p = a;` then one of:

expr	means	v	after
`v = *p++`	use *p, then move p	10	p → a[1], a unchanged
`v = (*p)++`	use *p, then bump a[0]	10	a[0] = 11, p still a
`v = ++*p`	bump a[0], then use	11	a[0] = 11
`v = *++p`	move p, then use	20	p → a[1]

Double pointers — the exam pattern (drawn step by step on p.6B)

```
int A[4]={1,2,3,4}; int *p[2], **q;       AY24/25 Q11
p[0]=&A[1]; p[1]=&A[3]; q=p;
*q++ = &A[2];  // p[0]=&A[2], THEN q -> p[1]
**q = 100;     // *p[1] = A[3] = 100
=> 1 2 3 100   (B)

p[0]=&A[0]; p[1]=&A[2]; q=p;              AY25/26 Q11
*q = &A[1];    // p[0]=&A[1]
**q++ = 100;   // **(old q) = A[1] = 100; q -> p[1]
=> 1 100 3 4   (C)
*q = p[1] = &A[2] = 0x1000 + 2*4 = 0x1008  (Q12: B)
q itself = 0x5004 (q moved by sizeof(int*) = 4)
```

Lecture trace (L4B): `a=8,b=15,c=23; p1=&b; p2=&c; p3=p2; *p1*=a; while(*p2>0) {*p2-=a;(*p1)++;}` ⇒ **1: 15 23 23 · 2: 123 −1 −1 · 3: 8 123 −1**. `int x=5,*p=&x,**q=&p; **q+=1;` ⇒ x = 6.

Pointer arithmetic & sizes

- `p + i` adds i × sizeof(*p) bytes: char 1, int 4, float 4, double 8, pointer 4 in the exam's 32-bit model (8 on a 64-bit machine), struct = sizeof(struct). L4B: ap++ +4, cp++ +1, dp++ +8; ap += 3 ⇒ +12.
- `q - p` (same array) = number of **elements**, not bytes.
- `a[i]` ≡ `*(a+i)`; `p[-1]` is fine if in bounds. `int a[]={10,20,30,40}, *p=a;` ⇒ `*(p+2)`=30, `*p+2`=12, `p[3]-*a`=30.
- An array name is a **constant** pointer to element 0: `a == &a[0]`; `a = b;` is illegal ⇒ copy with a loop.
- `sizeof` an array = total bytes; inside a function the parameter is a pointer, so sizeof gives 4/8. `char s[]="hello"`: sizeof 6, strlen 5; `char *t="hello"`: sizeof(t) = pointer size.
- `int *n; *n = 123;` compiles, then segfaults (n points nowhere). `double b = &a;` is an error.

Scope, lifetime, static (AY24/25 Q9, Q10)

```
int x = 1;                        // global
int main(void) {
  int x = 2;                      // hides global
  for (int x = 0; x < 4; x++) {   // loop x
    int x = 3;                    // NEW each iteration
    printf("%d ", x++);           // 3 3 3 3
  }
  printf("%d\n", x);              // 2  => "3 3 3 3 2" (A)
}
// with  static int x = 3;  inside the loop:
// initialised ONCE, survives iterations => "3 4 5 6 2" (D)
```

- Lexical scope: decided by the braces at compile time. A function sees globals, never its caller's locals (L4D: `foo()` prints the global −1 even when main's x = 30).
- Locals (automatic) die when the function returns; `static` locals live for the whole run; `static` on a global restricts it to the file.
- `for(int i=0;i<5;i++){ static int t=0; t+=i; }` ⇒ t = 10.

Increment & loop traps (verified)

code	output / result
`for(i=0;i<10;i++) printf("%d ",++i);`	**1 3 5 7 9** (both papers)
`i=5; do printf("%d ",i--); while(--i && i>0);`	**5 3 1** (AY25/26 Q10: B)
`i=0; while(i++<3);`	i = **4** (note the `;` body)
`i=0; do printf("%d",i); while(i-- > 0);`	prints **0**; i ends −1 (body always runs once)
`x=3; y=x++ + 10; z=++x + 10;`	x=5, y=13, z=15
`for(i=0;i<10;i++){ i++; }`	i = 10 after loop
`if (x = 0) … else …`	assignment! always else; `if (x = 5)` always true
`7/2, -7/2, -7%2, 7%-2, 7/2.0`	3, −3, −1, 1, 3.5 (truncate toward 0)
`float f = 5/2;`	2.00 (integer division first)
`unsigned u = -1;`	4294967295

Valid or not? (initialisers & statements)

`int A[4] = {1,2,3};`	✓ **A[3] = 0 (AY24/25 Q7: B)**
`int A[] = {1,2,3};`	✓ **size 3**
`int A[4] = {1,2,3,4,5};`	✗ **excess elements (warning)**
`int A[4] = [1,2,3,4]; / {1,,3,4};`	✗ **syntax**
`int f[5]; f = {..}; / f[5] = {..};`	✗ **braces only at declaration**
`char s[] = "abc";`	✓ **4 bytes incl. '\0'**
`char s[3] = "abc";`	✓ **in C — no '\0' stored, so not a string**
`char s[3] = "abcd";`	✗ **too long (warning)**
`struct{int x; char s[3]; int y;} stu = {1, "abc"};`	✓ **y = 0 (AY25/26 Q7: C)**
`… = {,"abc"}; / = {1,"abc",2,3};`	✗ **missing / excess**
`char name[10]; name = "Matthew";`	✗ **use strcpy**
`char *t; t = "Matthew";`	✓ **pointer to literal (don't modify)**
`if () / if (;) / if (i;)`	✗
`if (i ? 0 : 1) {…}`	✓ **AY25/26 Q8: D**
`struct copy: s2 = s1;`	✓ **copies every member (even arrays inside)**
`call before prototype`	✗ **implicit int → "conflicting types"**

Strings (L5B)

- A string is a char array ending in '\0' (0x00). strlen doesn't count it; printf("%s") and strlen run until they find it — without it you get garbage ("Length = 8, apple¿ø<").
- `scanf("%s")` stops at whitespace ("My book" → "My"); `fgets(s,n,stdin)` reads ≤ n−1 chars **and keeps the '\n'** ⇒ strip it; `puts` adds a newline.
- `strcmp`: <0, 0, >0 by ASCII. `strcpy` into too small a buffer overflows silently. `strncpy/strncmp` take n.
- Chars are small ints: `'a'+1` = 'b'; toupper = c−32 or c & 0xDF; digit = c−'0'.

Functions: what can a callee change?

passed	callee can change caller's data?
int / double / char (by value)	No — works on a copy (swap(a,b) fails: 2 3 stays 2 3)
&x (pointer)	Yes, via *p (swap(&a,&b) works)
array (decays to pointer)	**Yes** — elements (use `const` to forbid)
struct by value	No — whole struct copied ("Brusco 23" unchanged)
&struct	Yes via `p->m` ("Alexandra 25")

L4D: `f(a,b,c)` with a=9,b=−2,c=5 prints x=1,y=10,z=16 but a,b,c stay 9,−2,5; the pointer version changes them to 1,10,16. Returning a struct copies it back; typedef names a type but allocates nothing.

6B · Pointers drawn out, step by step

box = a variable · arrow = “holds the address of” · orange = what that line changed

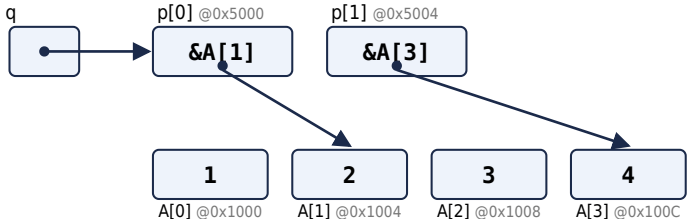
How to read * and ** in the pictures

expression	in the picture	expression	what it changes
q	q's arrow (which p box it points at)	q++	q's arrow moves to the next p box
*q	the p box q points at (itself a pointer)	(*q)++	that p box's arrow moves to the next A box
**q	follow 2 arrows: q → p box → A box	**q += 1	the A value goes up by 1
*q++ = X	write X into the p box, **then** move q	**q++ = X	write X into the A box, **then** move q

Postfix ++ belongs to q (it binds tighter than *), but it happens **after** the old q has been used. See the precedence table on p.6.

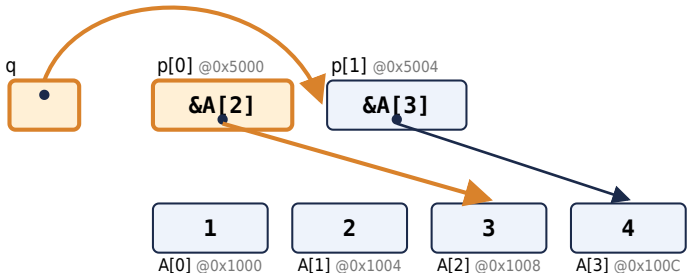
AY24/25 Q11 — int A[4]={1,2,3,4}; int *p[2], **q;

① p[0]=&A[1]; p[1]=&A[3]; q=p;



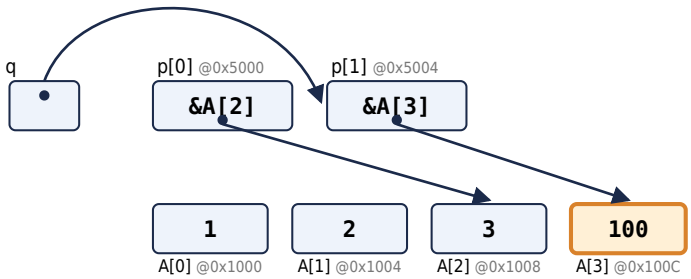
q=p makes q point at the first box of p, i.e. p[0].

② *q++ = &A[2];



***q** = the box q points at (p[0]). Store &A[2] there, so p[0]'s arrow moves to A[2]. **Then** the **++** moves q's arrow to the next box, p[1].

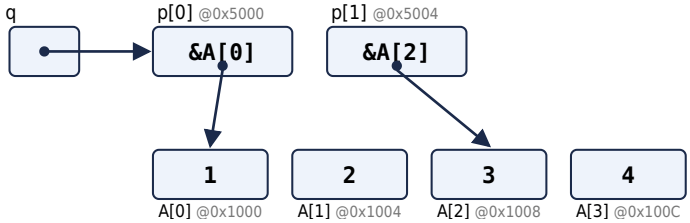
③ **q = 100;



****q** = follow two arrows: q → p[1] → A[3]. Store 100 there. **A = 1 2 3 100 ⇒ (B)**

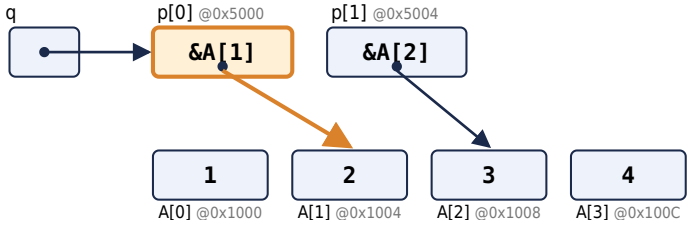
AY25/26 Q11 & Q12 — same declarations

① p[0]=&A[0]; p[1]=&A[2]; q=p;



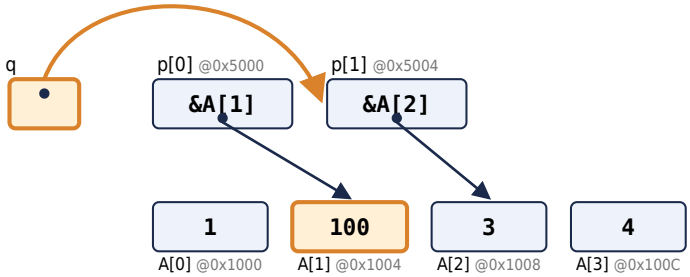
q points at p[0]; p[0] → A[0], p[1] → A[2].

② *q = &A[1];



***q** = p[0]. Store &A[1] there: p[0]'s arrow moves to A[1]. q does not move (no ++).

③ **q++ = 100;



Use the **old** q first: q → p[0] → A[1], store 100. **Then** ++ moves q to p[1]. **A = 1 100 3 4 ⇒ (C)**. Q12: *q is now p[1] = &A[2] = 0x1000 + 2×4 = **0x1008 (B)**; q itself = 0x5004.

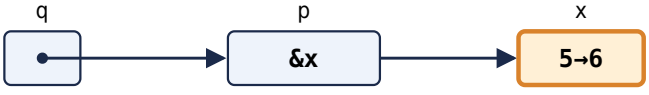
Lecture trace (L4B TracePointers.c)

```
int a = 8, b = 15, c = 23;
int *p1, *p2, *p3;
p1 = &b; p2 = &c; p3 = p2;   // p3 copies p2's arrow -> also c
printf("1: %d %d %d\n", *p1, *p2, *p3);
*p1 *= a;
while (*p2 > 0) { *p2 -= a; (*p1)++; }
printf("2: %d %d %d\n", *p1, *p2, *p3);
printf("3: %d %d %d\n", a, b, c);
```

after	a	b (= *p1)	c (= *p2 = *p3)	prints
setup	8	15	23	1: 15 23 23
*p1 *= a	8	120	23	
loop pass 1	8	121	15	
loop pass 2	8	122	7	
loop pass 3	8	123	−1 (stop)	2: 123 -1 -1
end	8	123	−1	3: 8 123 -1

p1 never moves (always → b) and p2, p3 both point at c, so changing *p2 also changes what *p3 shows. Writing through a pointer changes the **variable** it points at.

Pointer to a pointer: `int x=5, *p=&x, **q=&p; **q += 1;`



q = &p · *q = p (= &x) · **q = x

7 · MIPS ↔ C: compile patterns, decompiling tricks, bytes & endianness

all snippets simulated & checked

C → MIPS templates (invert the condition to skip the body)

```
# if (i == j) f = g + h;          # if (i==j) f=g+h; else f=g-h;
      bne  $s3,$s4,Exit                bne  $s3,$s4,Else
      add  $s0,$s1,$s2                 add  $s0,$s1,$s2
Exit:                                  j    Exit
                                 Else: sub  $s0,$s1,$s2
                                 Exit:
```

```
# while (j == k) i++;            # for (i=0; i<10; i++) a += 5;
Loop: bne  $s4,$s5,Exit                add  $s0,$zero,$zero
      addi $s3,$s3,1                   addi $s1,$zero,10
      j    Loop                  Loop: beq  $s0,$s1,Exit
Exit:                                  addi $s2,$s2,5
                                       addi $s0,$s0,1
                                       j    Loop
                                 Exit:
```

```
# for (i=0; i<n; i++)  (general <)   # do { body } while (x != 0);
      add  $t0,$zero,$zero       Loop:  ...body...
Loop: slt  $t1,$t0,$a1                 bne  $x,$zero,Loop
      beq  $t1,$zero,Exit
      ...body...                   # if (a && b) S;     # if (a || b) S;
      addi $t0,$t0,1                     beq a,$0,Skip        bne a,$0,Do
      j    Loop                          beq b,$0,Skip        beq b,$0,Skip
Exit:                                    S                 Do:  S
                                 Skip:                     Skip:
```

Branch-condition decoder

MIPS	goto L when…
`beq a,b,L / bne a,b,L`	a == b / a != b
`slt t,a,b ; bne t,$0,L`	a < b
`slt t,a,b ; beq t,$0,L`	a >= b
`slt t,b,a ; bne t,$0,L`	a > b
`slt t,b,a ; beq t,$0,L`	a <= b
`slti t,a,K ; bne / beq`	a < K / a >= K
`beq x,$0,L / bne x,$0,L`	!x / x (e.g. `while(*s)`)

If the branch **skips** the code below it, the C condition for running that code is the **opposite**: `beq $s1,$s2,Exit; add $s0,$0,$0` ⇒ `if (i != j) f = 0;` (L7 Ex 1).

Decompiling MIPS → C (Assg 1 Q4 style)

1. Mark labels. **Backward** branch/jump = loop; **forward** = if/skip.
2. Registers read before being written = parameters ($a0, $a1…) or inputs; **$v0** before `jr $ra` = return value ⇒ that gives the prototype.
3. A register used as a base in `lw/sw/lb/sb` is a **pointer**. Stepped by 4 with lw ⇒ `int*`; by 1 with lb/lbu ⇒ `char*`. `sll t,i,2; add t,base,t` ⇒ `A[i]`.
4. Rewrite as `if (...) goto` C first, then fold into while/for. Drop registers that only hold temporaries.
5. Recognise idioms (table below), then name variables by meaning, not by register.
6. Test with a tiny input (e.g. array {1,2,3}) to confirm.

Idioms at a glance

MIPS	C meaning
`sll t,i,2 ; add t,a,t ; lw x,0(t)`	x = a[i] (int)
`add t,a,i ; lb x,0(t)`	x = s[i] (char)
`lw x,12(a)`	x = a[3] (constant index ×4)
`sw x,0(p) ; addi p,p,4`	*p++ = x
`andi t,x,1`	x is odd (= x % 2 only when x ≥ 0)
`andi t,x,7 / srl t,x,3`	x % 8 / x / 8 (non-negative x)
`sra t,x,31`	0 if x ≥ 0, −1 if x < 0
`sub x,$0,y / nor x,y,$0`	x = −y / x = ~y
`xor t,a,b ; sltiu t,t,1`	t = (a == b)
`xor t,a,b ; sltu t,$0,t`	t = (a != b)
`addi t,x,-1 ; and t,t,x ; sltiu v,t,1`	x is a power of 2 (also says yes for 0!)
`sll t,x,3 ; sll u,x,1 ; add v,t,u`	v = x*10
`xor a,a,b ; xor b,a,b ; xor a,a,b`	swap a,b without temp
`slti t,c,97 … slti t,c,123 … addi c,c,-32`	'a' ≤ c ≤ 'z' ⇒ toupper

Worked mystery functions

```
      add  $v0,$zero,$zero       int f(char *s) {       // strlen
loop: lb   $t0,0($a0)              int n = 0;
      beq  $t0,$zero,done          while (*s) { s++; n++; }
      addi $a0,$a0,1               return n;
      addi $v0,$v0,1             }  // "CS2100" -> 6; 34 instr.
      j    loop                     // executed = 1 + 5n + 2 + 1(jr)
done: jr   $ra

      add  $v0,$zero,$zero       int g(unsigned x) {    // popcount
loop: beq  $a0,$zero,done          int c = 0;
      andi $t0,$a0,1               while (x) { c += x & 1; x >>= 1; }
      add  $v0,$v0,$t0             return c;
      srl  $a0,$a0,1             }  // srl (not sra): ends for -1 -> 32
      j    loop
done: jr   $ra

      add  $v0,$zero,$zero       int h(int *a, int n) { // count odd
      add  $t0,$zero,$zero         int c = 0;
loop: slt  $t1,$t0,$a1             for (int i=0; i<n; i++)
      beq  $t1,$zero,done            c += a[i] & 1;
      sll  $t2,$t0,2               return c;
      add  $t2,$a0,$t2           }  // {3,4,-3,7,10,-8} -> 3
      lw   $t3,0($t2)
      andi $t3,$t3,1
      add  $v0,$v0,$t3
      addi $t0,$t0,1
      j    loop
done: jr   $ra
```

```
# |x| without branches          # max(a,b) in $v0
sra $t1,$a0,31  # mask 0 / -1         slt $t0,$a0,$a1
xor $t0,$a0,$t1 # ~x if negative      beq $t0,$zero,L1
sub $t0,$t0,$t1 # +1 if negative      add $v0,$a1,$zero
                                       j   E
                                 L1:   add $v0,$a0,$zero
                                 E:
```

Bytes, halfwords, endianness

load	byte 0x9C	byte 0x41
`lb`	`0xFFFFFF9C (−100)`	`0x00000041`
`lbu`	`0x0000009C (156)`	`0x00000041`

`sb` stores only rt[7:0]; `sh` only rt[15:0]. Byte offsets need no alignment; lw/sw need addr%4 = 0, lh/sh even (else use ulw/usw pseudo-instructions, built from lb/sb).

sw 0x12345678 → 0x1000	0x1000	0x1001	0x1002	0x1003
Big-endian (MSB at lowest addr; IBM, Motorola, MIPS/SGI, SPARC)	12	34	56	78
Little-endian (LSB at lowest addr; x86, VAX, Alpha, the online MIPS interpreter)	78	56	34	12

So `lbu $t0,1($s0)` gives 0x34 (big) or 0x56 (little). Endianness never changes the *number of 1s* in a word ⇒ parity survives a big→little transfer (AY25/26 Q18: all three have an odd number of 1s ⇒ D).

Bit manipulation (watch zero-extension!)

- Set bit k (k<16): `ori $t0,$t0,(1<<k)`. Toggle: `xori`. Test: `andi $t1,$t0,(1<<k)` then bne/beq $0.
- **Clear bit 0 of the whole word:** `andi $t0,$t0,0xFFFE` also clears bits 31–16! Correct: `addi $t1,$zero,-2` (0xFFFFFFFE) then `and $t0,$t0,$t1`.
- AY25/26 Q17 (set bits 9, 12, 15, rest 0 ⇒ 0x9200): (i) ORs into an unknown $t0; (ii) addi sign-extends 0x9200 → 0xFFFF92FF; (iii) constant 0x92 is wrong; (iv) gives 0x12400 ⇒ **E**. The right answer is simply `ori $t0,$zero,0x9200`.
- Get byte k of a word: `srl $t,$x,8k ; andi $t,$t,0xFF`.

Structs & 2-D arrays in MIPS Field address = base + offset of field. `typedef struct {int x; int y;} pt; pt a[]`: `a[i].y` is at a + 8i + 4 ⇒ `sll t,i,3; add t,a,t; lw v,4(t)`. `p->y` with p in $a0 ⇒ `lw v,4($a0)`. (A char member followed by an int is padded so the int starts at a multiple of 4.) Row-major `int m[R][C]`: m[i][j] at base + 4(i·C + j).

Answer-writing mistakes that cost marks

- Using `lw` on a char array (use lb/lbu, step 1) or stepping an int array by 1 (step 4).
- Clobbering an input register the question says to preserve; using $t registers when told to use only $at.
- Pseudo-instructions when the question says “real / core instructions only”.
- Forgetting that `addi` sign-extends (0x8000–0xFFFF are negative) and `andi/ori` zero-extend.
- Off-by-one in loop bounds when translating < vs <=: swap slt operands or use slti with K+1.

Counting executed instructions

- Total = (setup before loop) + iterations × (instructions per pass, including the branch/jump) + (instructions on the exit path).
- Top-tested loop (branch at top + `j` at bottom) pays 2 control instructions per pass; bottom-tested (single bne) pays 1.
- Distinguish *executed* vs *taken*: L7 Ex 5 — bne executed 10 times, taken 9; ulw from $t0+10 down to $t0+1 touches bytes +1…+13 ⇒ 13 unique bytes.
- L7 Ex 3: `beq $t1,$zero,Loop` after decrementing 20 → 19 falls through: only **6** instructions run, $t2 = 20. Read branch sense carefully.
- `addi $t0,$0,0; addi $t1,$0,5; L: addi $t0,$t0,2; addi $t1,$t1,-1; bne $t1,$0,L` ⇒ 2 + 5×3 = **17** instructions, $t0 = 10.

8 · Datapath & control (L11–L12)

Main control truth table (lecture) + derived rows

instr	opcode	RegDst	ALUSrc	MemToReg	RegWrite	MemRead	MemWrite	Branch	ALUop	ALUcontrol
R-type	000000	1	0	0	1	0	0	0	10	from funct
lw	100011	0	1	1	1	1	0	0	00	0010
sw	101011	X	1	X	0	0	1	0	00	0010
beq	000100	X	0	X	0	0	0	1	01	0110
*addi**	001000	0	1	0	1	0	0	0	00	0010

*addi is derived (works on the lecture datapath with ALUop 00). X = don't care because nothing is written (RegWrite = 0). **Not supported** by the lecture datapath: j (no jump path), andi/ori (sign-extender is always on), sll/srl (no shamt path), bne (no inverted Zero).

Multiplexers

signal	0 selects	1 selects
RegDst	rt = Inst[20:16]	rd = Inst[15:11]
ALUSrc	Read data 2 ([rt])	SignExt(Inst[15:0])
MemToReg	ALU result	Memory read data
PCSrc	PC + 4	branch target

MemToReg's mux is **drawn** with input 1 on top ("swapped") only so the wires don't cross (AY24/25 Q24: C). The meaning never changes. PCSrc = Branch AND isZero.

ALU control

ALUcontrol	op	Ainv Binv Op
0000	AND	0 0 00
0001	OR	0 0 01
0010	add	0 0 10
0110	sub	0 1 10
0111	slt	0 1 11
1100	NOR	1 1 00

Two-level decoding: opcode (6) → main control → ALUop (2): 00 lw/sw (add), 01 beq (subtract), 10 R-type (look at funct). ALU control unit: **8 inputs** (ALUop 2 + funct 6) → **4 outputs**. funct: add 100000, sub 100010, and 100100, or 100101, slt 101010.
Equations: ALUctrl3 = 0; ALUctrl2 = ALUop0 + ALUop1·F1; ALUctrl1 = ¬ALUop1 + ¬F2; ALUctrl0 = ALUop1·(F3 + F0). RegDst = R; ALUSrc = lw + sw; MemToReg = MemRead = lw; RegWrite = R + lw; MemWrite = sw; Branch = ALUop0 = beq; ALUop1 = R.
1-bit ALU: Ainvert/Binvert invert the inputs, Operation picks AND/OR/adder/less. For subtract, Binvert = 1 and the LSB's carry-in = 1. NOR = (¬A)·(¬B), hence Ainv = Binv = 1 with AND.

"Every wire has a value" — fill-in template

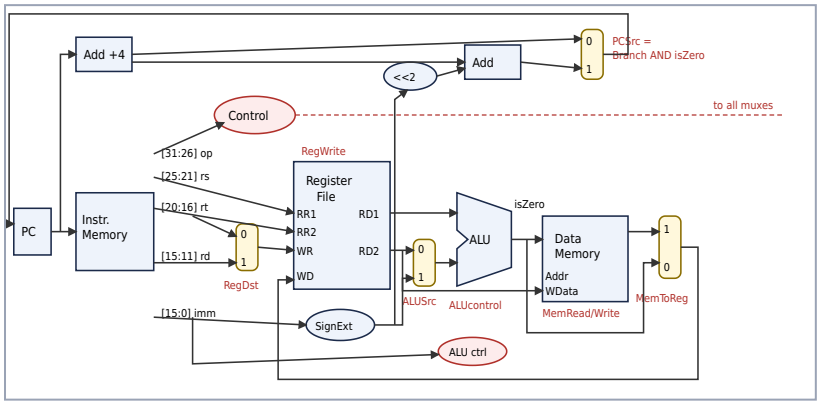
point	value	point	value
RR1 / RR2	rs / rt (always, even if unused)	Sign-extend out	SE(Inst[15:0]) — also for R-type
WR	RegDst ? rd : rt (exists even when RegWrite=0)	ALU in 1	[rs]
RD1 / RD2	[rs] / [rt]	ALU in 2	ALUSrc ? SE(imm) : [rt]
WD (reg)	MemToReg ? Mem[ALU] : ALU result	DM address / write data	ALU result / [rt]
Branch target	PC+4 + SE(imm)<<2 (computed every cycle)	next PC	PCSrc ? target : PC+4

```
add $t4,$v0,$s1 = 0x00516020, $t4=0x100 $v0=0x200 $s1=0x300   (AY25/26 Q20-25)
RR1 = 2 (A), RR2 = 17 (C), WR = 12 (B), SignExt out = 0x00006020 (B, unused)
ALU in2 = [$s1] = 0x300 (D), result 0x500; RegDst=1 RegWrite=1 ALUSrc=0
MemRead=0 MemWrite=0 MemToReg=0 (B)

0x10000103 at PC 0x491 = beq $0,$0,0x103                          (AY24/25 Q28)
RR1 = $0, WR = $0 (rt and rd fields are both 0), WD = 0 or garbage (not written)
ALU in1 = [$0] = 0, taken -> PC = 0x495 + 0x40C = 0x8A1

sw $t1,8($t2) = 0xAD490008: RR1=$t2, RR2=$t1, WR=$t1 or $0 (X), ALU = [$t2]+8,
DM write data = [$t1], MemWrite=1, RegWrite=0, ALUop=00
```

Simplified datapath (mux input numbers as in lecture)



Fault injection: which instructions break?

fault	breaks	still fine
RegDst stuck 0	R-type (writes rt)	lw, addi, sw, beq
RegDst stuck 1	lw, addi (write to imm[15:11])	R, sw, beq
ALUSrc stuck 0	lw, sw, addi (use [rt] not imm)	R, beq
ALUSrc stuck 1	R-type, beq (compare rs with imm)	lw, sw, addi
MemToReg stuck 0 / WD = ALU	**lw** (AY24/25 Q22: C)	R, addi, sw, beq
MemToReg stuck 1	R-type, addi	lw, sw, beq
RegWrite stuck 0	R, lw, addi	sw, beq
RegWrite stuck 1	sw, beq (corrupt a register unless it is $0)	R, lw, addi
MemWrite stuck 1	everything except sw corrupts memory	sw
Branch stuck 0	beq never taken	rest
Branch stuck 1	any instr whose ALU result is 0 jumps	—
ALUop stuck 00	sub/and/or/slt; beq (adds)	add, lw, sw, addi
Zero-extend instead of sign-extend	negative imm: addi −50, lw −8($sp), backward beq	andi/ori would now work

Critical path (AY25/26 Q28 method)

Inst-Mem	Adder	MUX	ALU	Reg-File	Data-Mem	Ctrl / ALU-ctrl	Shift / SignExt / AND
400	100	30	120	200	350	100	20

Method: ready time of a block = max(ready times of all its inputs, data **and** control) + its own delay. Common part: IM 400 → SignExt 420, Control 500, RegFile read 600, ALU-control max(500, funct 400)+100 = 600, RegDst mux 530.

instr	path	total
addi (key)	ALUSrc mux max(420,500)+30 = 530; ALU max(600,530,600)+120 = 720; MemToReg mux 750; RegFile write +200	**950 ps**
R-type†	ALUSrc mux now waits for RD2: 600+30 = 630; ALU 750; mux 780; write	980 ps
lw†	ALU 720; Data-Mem 1070; MemToReg 1100; write	**1300 ps**
sw†	ALU 720; Data-Mem write	1070 ps
beq†	ALU 750 (RD2 via mux); AND 770; PCSrc mux 800 (target ready at 420+20+100 = 540)	800 ps

†My derivations with the same model, not from a key. Clock period for single-cycle = slowest (lw). Key's addi answer: Inst-Mem, Reg-File (twice), ALU, MUX on the path; Adder, Data-Mem, Control, Shift-left not on it (two routes to the ALU tie at 720).

Lecture timing & concepts

- L12: memory 2 ns, ALU/adders 2 ns, register file 1 ns ⇒ ALU-type 6, lw 8, sw 7, beq 5 ns. Single-cycle forces every instruction to 8 ns. Fixes: multicycle (steps of 1 cycle each) or pipelining.
- Stages: Fetch → Decode (+ operand fetch) → ALU → Memory (lw/sw only) → Register write. Only lw uses all five.
- Clocking: state (PC, RegFile, Data-Mem) is read during the cycle and written on the **rising edge** at its end (AY24/25 Q23: D).
- RegFile: 2 reads + 1 write per cycle; 5-bit register numbers, 32-bit data. Data-Mem: MemRead/MemWrite never both 1.
- True/false bank: RR2 is 5 bits ✓; RD2 used by all instructions ✗ (lw, addi don't use it); ALU combinational ✓; $zero ignores writes ✓; PC 36-bit ✗ (32); ALU-control 6-in/4-out ✗ (8 in); ALUop controls RegDst ✗; all opcodes 6 bits ✓; funct sometimes sets ALUcontrol ✓; beq ALUop = 01 ✓.

Extending the datapath (what to add)

- **bne:** new BNE signal; PCSrc = Branch·Zero + BNE·¬Zero.
- **j:** JumpAddr = {PC+4[31:28], Inst[25:0], 00}; a mux after the PCSrc mux; Jump signal.
- **andi/ori:** zero-extender + ExtOp mux; ALUop must encode AND/OR from the opcode.
- **sll/srl:** shamt (Inst[10:6]) to the ALU or a shifter; ALU needs a shift operation.
- **lui:** imm<<16 path into the MemToReg mux (3 inputs).
- **jal:** write PC+4 to $31: RegDst mux gets constant 31, MemToReg mux gets PC+4.

9 · ISA design (L10) & expanding opcodes — plus corrections

formulas checked by brute force over every allocation

RISC vs CISC

CISC (x86/IA-32, VAX): one instruction does complex work (VAX could multiply polynomials); smaller programs when memory was expensive; complex hardware, little room to optimise. **RISC** (MIPS, ARM): small, simple instruction set that is easy to build and optimise; software (compiler) combines simple instructions.

Concept 1 — storage architectures (C = A + B)

Stack	Accumulator	Load-store (reg-reg)	Memory-memory

`Push A`
`Push B`
`Add`
`Pop C`
	`Load A`
	`Add B`
	`Store C`
		`Load R1,A`
		`Load R2,B`
		`Add R3,R1,R2`
		`Store R3,C`
			`Add C,A,B`

operands implicit on top of stack	one operand implicit in the accumulator (IBM 701, PDP-8)	only explicit register operands; only load/store touch memory (MIPS, Alpha)	all operands in memory (VAX)

Register-memory (one operand may be in memory): Motorola 68000, Intel 80386. GPR architectures won because CPUs outpaced memory, multicore needs fewer memory trips, and compilers improved. RISC = load-store; CISC mixes reg-reg and reg-mem (for compatibility).
Variant D = (A+B)×C: stack Push A, Push B, Add, Push C, Mul, Pop D (6); accumulator Load A, Add B, Mul C, Store D (4); load-store Load R1,A; Load R2,B; Add R3,R1,R2; Load R4,C; Mul R5,R3,R4; Store R5,D (6); mem-mem Add D,A,B; Mul D,D,C (2).

Concept 2 — memory & addressing modes

- k-bit address ⇒ 2^k addressable locations; each transfer is one n-bit word over an n-bit data bus (via MAR/MDR).
- **Endianness:** big-endian = MSB at the lowest address; little-endian = LSB at the lowest address. MIPS endianness is implementation-specific.

mode	example	meaning
Register	`Add R4,R3`	R4 ← R4+R3
Immediate	`Add R4,#3`	R4 ← R4+3
Displacement	`Add R4,100(R1)`	R4 ← R4+M[100+R1]
Register indirect	`Add R4,(R1)`	R4 ← R4+M[R1]
Indexed / base	`Add R3,(R1+R2)`	R3 ← R3+M[R1+R2]
Direct / absolute	`Add R1,(1001)`	R1 ← R1+M[1001]
Memory indirect	`Add R1,@(R3)`	R1 ← R1+M[M[R3]]
Auto-increment	`Add R1,(R2)+`	R1 ← R1+M[R2]; R2 ← R2+d
Auto-decrement	`Add R1,-(R2)`	R2 ← R2−d; R1 ← R1+M[R2]
Scaled	`Add R1,100(R2)[R3]`	R1 ← R1+M[100+R2+R3·d]

MIPS operand modes: register, immediate, displacement (L10) + PC-relative and pseudo-direct for control flow (L8). A MIPS `lw $t0,0($t1)` is register-indirect as a special case of displacement.

Concept 3 — operations & frequency

Categories: data movement, arithmetic, logical, shift, control flow, subroutine linkage, interrupt, synchronisation (atomic test&set), string, graphics. Top 10 integer instructions: load 22%, conditional branch 20%, compare 16%, store 12%, add 8%, AND 6%, sub 5%, move reg-reg 4%, call 1%, return 1% (96%). “Make the common case fast” (Amdahl): halving the time of the top 70% while slowing the other 30% by 50% gives 0.35 + 0.45 = **0.80** of the original time.

Concept 4 — instruction formats

Variable length (x86 1–17 bytes, VAX 1–54): compact and flexible, but multi-step fetch/decode. **Fixed length** (MIPS, PowerPC: 4 bytes): easy fetch/decode, simpler pipelining; bits are scarce. **Hybrid**: a mix. Typical operand types: char 8, half 16, word 32, single 32, double 64 bits; a 32-bit ISA should support 8/16/32-bit ints and 32/64-bit floats.

Concept 5 — encoding: expanding opcodes

Fixed length + different operand needs ⇒ give short opcodes to types with many operand bits and extend the opcode (using unused slots) for types with fewer operand bits. MIPS funct is an example. Start with the most constrained type (most operand bits). Main drawback of an aggressive scheme: **complex decoding logic** (AY25/26 Q19: A).

Recipe

1. For each type: opcode bits = instruction length − operand bits (per length if the ISA mixes lengths).
2. Sort types by opcode length: a (shortest) < a+b < a+b+c. So b, c are the *extra* bits at each tier.
3. **Maximum** (each type ≥ k_i): short-opcode types take only their minimum; every other slot expands.
   Max = $N_A + N_B + ((2^a - N_A)\cdot 2^b - N_B)\cdot 2^c$ (usually $N_A = N_B = 1$)
4. **Minimum, all encodings used:** each tier gives exactly **one** slot down; the last type fills its whole sub-space.
   Min = $(2^a - 1) + (2^b - 1) + 2^c$
5. Two types only: max = $1 + (2^a-1)\cdot 2^b$; min = $(2^a-1) + 2^b$.

Every exam question, solved with the recipe

question	setup	answer
AY24/25 Q17	R-type int: opcode 0x00, 6-bit funct	2^6 = **64** (B)
AY24/25 Q18	shifts moved to I-type ⇒ shamt (5) joins funct ⇒ 11 bits; FP R-type unchanged	2^{11} = **2048** (C)
AY24/25 Q19	16-bit; A 3×3-bit ops ⇒ 7-bit op; B 2×3 ⇒ 10; C 1×3 ⇒ 13 ⇒ a=7, b=3, c=3	2 + (127·8−1)·8 = **8122** (D)
AY24/25 Q20	same, minimum, all opcodes used	127 + 7 + 8 = **142** ⇒ E (key says 198, see below)
AY24/25 Q27	32-bit, 64 regs (6-bit fields), 16-bit imm ⇒ I-type op = 32−16−12 = 4 bits; R: 4 + 18 + 10-bit funct; J: 2 slots, 28-bit address (only 2 PC bits used). 16 slots = 10 I + 2 J + 4 R	R = $4\cdot 2^{10}$ = 4096; I = 10; total **4108**
AY25/26 Q27	64 regs. A: 16-bit, 2 regs ⇒ 4-bit op. C: 32-bit, 2 regs + 14 imm ⇒ 6-bit op. B: 32-bit, 3 regs + 6 shamt ⇒ 8-bit op. Tiers 4→6→8: a=4, b=2, c=2	max 2 + (15·4−1)·4 = **238**; min 15 + 3 + 4 = **22**
Variant	12-bit; 4-bit regs; 2-reg (4-bit op), 1-reg (8), 0-reg (12): a=b=c=4	max 3826, min 46
Variant	AY24/25 Q19 but ≥ 5 type-A and ≥ 5 type-B	10 + (123·8−5)·8 = **7842**

Related “what if”s: 64 registers with 16-bit imm in 32 bits ⇒ I-type opcode can be at most 4 bits. 16 registers ⇒ 4-bit fields ⇒ opcode could grow to 8 bits. If branch offsets counted bytes instead of words: reach $\pm 2^{15}$ bytes instead of $\pm 2^{17}$.

Corrections to the NotebookLM cheat sheet

1. **`struct {int x; char s[3]; int y;} stu = {1,"abc"};`** is **valid** (AY25/26 Q7 key: C). The sheet calls it a compile error. "abc" exactly fills s[3] with no '\0'; y = 0.
2. **MemToReg never changes meaning**: 0 = ALU result, 1 = memory data. Only the drawing swaps the inputs (to avoid crossing wires). The sheet's “if reversed, 0 becomes memory” is wrong.
3. **“Minimise: leave one free slot, lower classes get 1 each”** is wrong when all opcodes must be used: the last type fills its whole sub-space. Use Min = $(2^a-1)+(2^b-1)+2^c$.
4. The row `p[1]=&(A[2]); q=&p; (*q)++` is not from any paper; with `int *p[2]`, `(*q)++` increments an array ⇒ compile error. Use the real Q11 patterns (page 6).
5. Its control table includes **j**, which the lecture datapath does not support; and it writes sw's ALUOp as 0010 (that is ALUcontrol; ALUop = 00). It also omits Branch and MemRead.

Where the draft answer keys look wrong

- **AY24/25 Q20:** key says 198 (A) by giving B and C their own top-level slots. Nesting is smaller: 127 A + 7 B + 8 C = **142** (verified by brute force) ⇒ E. The AY25/26 Q27 key uses exactly this nested method (22).
- **AY24/25 Q14:** question paper option D = **xori**, which works after lui. The key's copy shows D = nor and answers E.
- **AY25/26 Q26:** first line `andi $at, $rx, 0x1F` should read `$rz`.
- **AY25/26 Q28:** key says MUX is on the critical path “twice” but its total counts one mux (950 ps). The total is consistent with its diagram.

If one of these comes up, write a one-line justification next to your answer where the format allows.

10 · Practice bank (52 variants with answers)

cover the blue answers and try first · all values computed/verified

Numbers

1. Read 1001 1100 as unsigned, sign-magnitude, 1's, 2's, excess-128.
→ 156, −28, −99, −100, +28.

2. 8-bit 1's complement: 0000 0111 + 1111 1101.
→ 1|0000 0100, end-around carry → 0000 0101 = 5 (7 + −2).

3. 6-bit 2's complement: 011010 + 010011. Result? Overflow?
→ 101101 = −19; overflow (26 + 19 = 45 > 31, pos + pos = neg).

4. Sign-extend 4-bit 1011 to 8 bits in S-M, 1's, 2's.
→ 1000 0011 (−3), 1111 1011 (−4), 1111 1011 (−5).

5. $121_b = 144_{10}$. Find b.
→ $(b+1)^2 = 144 \Rightarrow b = 11$.

6. Largest 12-bit 2's-comp value in hex? Most negative 8-bit 1's-comp value?
→ 0x7FF (2047). 1000 0000 = −127.

7. 8-bit 2's-comp fixed point, 3 fraction bits: 1110 1100 = ? Precision?
→ $-20/8 = -2.5$. Precision $2^{-3} = 0.125$.

8. 4-digit 9's and 10's complement of 0123.
→ 9876 and 9877.

9. Why can 2's complement hold one more negative than positive?
→ Only one zero; 100…0 = -2^{n-1} has no positive partner (negating it gives itself).

IEEE 754 & L9

10. Encode −12.5.
→ 1.1001×2^3, E = 130 ⇒ 0xC1480000.

11. Decode 0x42C80000 and 0xC0A00000.
→ 100.0 and −5.0.

12. What is 0x00400000?
→ Denormal: $0.1_2 \times 2^{-126} = 2^{-127}$.

13. Which convert exactly: 16777217, 16777218, 33554432, 0x0120101D?
→ 16777218 and 33554432 only (spread of 1s ≤ 23).

14. FP16: bias, 1.0, max? BF16 1.0?
→ 15, 0x3C00, 65504 (0x7BFF). BF16 1.0 = 0x3F80.

15. FP8 1.0 in E4M3 and E5M2?
→ Bias 7: 0x38. Bias 15: 0x3C.

16. Round to nearest even: $2^{24}+3$ in FP32?
→ Tie between $2^{24}+2$ and $+4$ ⇒ the even one, $2^{24}+4$.

17. INT8-quantise 5.72 when max|x| = 12.47.
→ scale 10.18 ⇒ round(58.2) = 58.

18. 0x80000000 as float and as int?
→ −0.0 and -2^{31}.

19. Is (a+b)+c = a+(b+c) for floats?
→ Not in general: FP32 (1e8 + −1e8) + 1 = 1, but 1e8 + (−1e8 + 1) = 0.

C

20. `int a[]={10,20,30,40},*p=a;` print `*(p+2), *p+2, p[3]-*a`.
→ 30 12 30.

21. `i=0; while(i++<3); printf("%d",i);`
→ 4.

22. `i=0; do printf("%d",i); while(i-- > 0);` Output and final i?
→ "0", i = −1.

23. `char s[]="hello";` sizeof(s), strlen(s)?
→ 6, 5.

24. `x=3; y=x++ + 10; z=++x + 10;`
→ x = 5, y = 13, z = 15.

25. `int *p[2], **q=p;` p at 0x5000 (32-bit). After q++, q = ?
→ 0x5004.

26. −7/2 and −7%2 in C?
→ −3 and −1 (truncate toward 0).

27. `char s[3]="abc"; printf("%s",s);`
→ Compiles, but s has no '\0' ⇒ prints past the array (undefined).

28. A function sets `player.age = 25` on a struct passed by value. Caller sees?
→ The old age — the struct was copied. Pass &player and use ->.

MIPS

29. Encode `sw $ra,-8($sp)`.
→ 0xAFBFFFF8.

30. Decode 0x02328020, 0x8E080004, 0x00084080.
→ add $s0,$s1,$s2 · lw $t0,4($s0) · sll $t0,$t0,2.

31. beq at 0x00400020 with imm 0xFFFE? with 0xFFFF?
→ 0x0040001C. 0xFFFF branches to itself.

32. Where can a j at 0x0FFFFFFC go?
→ Only 0x10000000–0x1FFFFFFC (PC+4 is in the next block).

33. Furthest forward and backward branch targets?
→ PC + 0x20000 and PC − 0x1FFFC.

34. Byte 0x9C loaded with lb and lbu?
→ 0xFFFFFF9C (−100) and 0x0000009C (156).

35. sw 0x12345678 at 0x1000, then lbu from 0x1001?
→ Big-endian 0x34, little-endian 0x56.

36. $t0 = ($a0 == $a1) without branches.
→ `xor $t0,$a0,$a1; sltiu $t0,$t0,1`.

37. `sltiu $t0,$t1,-1` gives?
→ 1 unless $t1 = 0xFFFFFFFF (imm sign-extends, then unsigned compare).

38. $a0 = −1, $a1 = 1: slt vs sltu?
→ slt 1, sltu 0.

39. Why is `andi $t0,$t0,0xFFFE` a bad way to clear bit 0?
→ Zero-extension also clears bits 31–16.

40. Popcount loop (p.7) with `sra` instead of `srl`, input −1?
→ Never terminates: −1 sra 1 = −1.

41. `addi $t0,$0,0; addi $t1,$0,5; L: addi $t0,$t0,2; addi $t1,$t1,-1; bne $t1,$0,L`: count and $t0?
→ 17 instructions, $t0 = 10.

Datapath & ISA

42. Control signals for `sw $t1,8($t2)`.
→ RegDst X, ALUSrc 1, MemToReg X, RegWrite 0, MemRead 0, MemWrite 1, Branch 0, ALUop 00.

43. ALUSrc stuck at 1: what breaks?
→ R-type and beq.

44. lw and R-type latency with the AY25/26 Q28 table?
→ 1300 ps and 980 ps (clock = 1300 ps).

45. What does the ALU compute for beq, and when is it taken?
→ rs − rt (ALUcontrol 0110); taken when Zero = 1 and Branch = 1.

46. slt ALUcontrol and its Ainvert/Binvert/Operation bits?
→ 0111 = 0, 1, 11.

47. Why can't the lecture datapath run ori?
→ The immediate is always sign-extended and ALUop cannot ask for OR from an I-type opcode.

48. Sign-extend output while executing `add $t4,$v0,$s1`?
→ 0x00006020 (computed, then ignored by the ALUSrc mux).

49. D = (A+B)×C: instruction count on stack / accumulator / load-store / mem-mem?
→ 6 / 4 / 6 / 2.

50. 12-bit ISA, 4-bit registers; 2-reg, 1-reg, 0-reg types. Max and min (all used)?
→ 3826 and 46.

51. `Add R1,@(R3)` means?
→ Memory indirect: R1 ← R1 + M[M[R3]].

52. 64 registers, 16-bit immediate, 32-bit I-type: max opcode width?
→ 32 − 16 − 6 − 6 = 4 bits.

Exam-room tactics (from both papers)

- "None of the above" was correct several times (AY24/25 Q14, Q25; AY25/26 Q17). Don't avoid E.
- "Which is NOT valid": test every option fully before answering.
- Decode to binary once, split fields with the p.2 grids, and write the fields above the hex.
- For datapath fill-ins, give a value for every box even if the signal is unused ("0 or random" was accepted for WD).
- Short-answer MIPS: state assumptions and which registers you preserve.